

DSA

Data Structures & Algorithms

Apuntes para entrevistas Google-level

 Big-O	 Arrays	 Listas	 Árboles
 Grafos	 Búsqueda	 Sorting	 DP

Aldo Zepeda Montoya

zetinaa3@gmail.com

github.com/AldoZM

30 de junio de 2026

Índice general

I	 Fundamentos	4
1.	 Big-O Notation	5
2.	 Complejidad Espacial (Space Complexity)	9
II	 Estructuras Lineales	11
3.	 Arrays (Vectores)	12
4.	 Linked Lists (Listas Enlazadas)	15
5.	 Fast & Slow Pointers (Tortuga y Liebre)	19
6.	 Stacks (Pilas)	22
7.	 Queues (Filas / Colas)	25
III	 Estructuras No Lineales	28
8.	 Binary Search Trees (Árboles Binarios de Búsqueda)	29
9.	 AVL Trees (Árboles Binarios Balanceados)	33
10.	 Recorridos de Árboles (Tree Traversals)	37
11.	 LCA y Serialización de Árboles	41
12.	 Heaps (Montículos)	45
13.	 Hash Tables (Tablas Hash / Mapas)	48
14.	 Hashing a Fondo: Colisiones e Implementación	51
15.	 Patrones con Hash Map (Frecuencias, Prefix-Sum, Agrupación)	55
16.	 LRU Cache (Hash Map + Lista Doble)	59
17.	 Tries (Árboles de Prefijos)	63
18.	 Union-Find (Disjoint Set Union - DSU)	67

IV	 Grafos	71
19.	 Representación de Grafos	72
20.	 BFS y DFS (Recorridos de Grafos)	75
21.	 BFS Avanzado (Grid, Multi-source y 0-1 BFS)	79
22.	 Algoritmo de Dijkstra (Camino más Corto)	83
23.	 Bellman-Ford (Camino más Corto con Pesos Negativos)	87
24.	 Floyd-Warshall (Todos los Pares)	91
25.	 Topological Sort (Ordenamiento Topológico)	94
26.	 MST: Kruskal y Prim (Árbol de Expansión Mínima)	98
27.	 Detección de Ciclos en Grafos	102
28.	 Componentes Fuertemente Conexas (SCC · Kosaraju)	105
V	 Algoritmos de Ordenamiento	109
29.	 Ordenamientos Básicos (Bubble, Insertion, Selection)	110
30.	 Merge Sort (Ordenamiento por Mezcla)	114
31.	 Quick Sort (Ordenamiento Rápido)	117
VI	 Algoritmos de Búsqueda	120
32.	 Linear Search (Búsqueda Lineal)	121
33.	 Binary Search (Búsqueda Binaria)	124
VII	 Patrones de Entrevista	127
34.	 Two Pointers (Dos Punteros)	128
35.	 Sliding Window (Ventana Deslizante)	131
36.	 Introducción a Dynamic Programming (DP)	134
37.	 DP - Memoization (Top-Down)	137
38.	 DP - Tabulation (Bottom-Up)	140
39.	 Backtracking (Vuelta Atrás)	143



Fundamentos

Capítulo 1

Big-O Notation

¿Qué es?

Imagínate que tienes una caja de chocolates y quieres encontrar uno de sabor fresa. ¿Cuánto tiempo tardarías?

- **Caso 1:** Si sabes que el de fresa SIEMPRE está en la posición 1 → Lo agarras directo. No importa si hay 10 o 1,000,000 chocolates. → Siempre es 1 paso. ¡Rápidísimo!
- **Caso 2:** Si los chocolates están desordenados y tienes que revisar uno por uno hasta encontrar el de fresa → Si hay 10 chocolates, revisas hasta 10. → Si hay 1,000,000 chocolates, revisas hasta 1,000,000. → El tiempo crece igual que la cantidad de chocolates.
- **Caso 3:** Si por cada chocolate que revisas, tienes que revisar TODOS los demás para compararlos → Si hay 10 chocolates → $10 \times 10 = 100$ comparaciones → Si hay 100 chocolates → $100 \times 100 = 10,000$ comparaciones → ¡Crece muy rápido y se pone muy lento muy pronto!

Big-O Notation es la forma que tienen los programadores para describir qué tan rápido o lento crece el tiempo que tarda un algoritmo cuando le das más y más datos.

La "n" siempre representa la cantidad de datos de entrada. Big-O siempre describe el PEOR caso posible (lo más lento que puede ser).

Dicho de otra forma

Big-O es la métrica estándar para comparar la eficiencia de algoritmos independientemente del hardware. Describe cómo escala el tiempo de ejecución (o uso de memoria) en función del tamaño del input "n", siempre en el worst case.

En interviews, SIEMPRE tienes que dar el Big-O de tu solución. Es tan esperado como que el código funcione.

Diagrama

Tiempo





$O(1)$	→ Constante	→ IDEAL
$O(\log n)$	→ Logarítmico	→ MUY BUENO
$O(n)$	→ Lineal	→ ACEPTABLE
$O(n \log n)$	→ Linealítmico	→ BUENO para sorting
$O(n^2)$	→ Cuadrático	→ MALO (evitar si puedes)
$O(2^n)$	→ Exponencial	→ FATAL (solo para n muy pequeño)

⚙️ ¿Cómo funciona?

$O(1)$ — Tiempo Constante Sin importar cuántos datos tengas, siempre tarda lo mismo. Ejemplo: acceder a `arr[5]` directamente por índice.

$O(\log n)$ — Tiempo Logarítmico Cada paso **DIVIDE** el problema a la mitad. Si tienes 1,024 elementos → necesitas máximo 10 pasos ($2^{10} = 1024$). Ejemplo: Binary search.

$O(n)$ — Tiempo Lineal Si duplicas el input, el tiempo se duplica. Ejemplo: recorrer todos los elementos de una lista una vez.

$O(n \log n)$ — Tiempo Linealítmico Un poco peor que lineal. La mayoría de buenos algoritmos de sorting. Ejemplo: Merge Sort.

$O(n^2)$ — Tiempo Cuadrático Dos loops anidados sobre el mismo input. Si duplicas el input, el tiempo se **CUADRUPLICA**. Ejemplo: Bubble Sort.

$O(2^n)$ — Tiempo Exponencial Cada elemento nuevo **DUPLICA** el trabajo. $n=30$ → más de 1,000,000,000 operaciones. Ejemplo: Fibonacci sin memoization.

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // O(1) - Constante
6 int obtenerPrimero(vector<int>& arr) {
7     return arr[0]; // acceso directo por índice → O(1)
8 }
9
10 // O(n) - Lineal
11 bool buscarLineal(vector<int>& arr, int objetivo) {
12     for (int i = 0; i < arr.size(); i++) {

```

```

13         if (arr[i] == objetivo) return true;
14     }
15     return false;
16 }
17
18 // O(n^2) - Cuadrático
19 bool tieneDuplicados(vector<int>& arr) {
20     int n = arr.size();
21     for (int i = 0; i < n; i++) {
22         for (int j = i + 1; j < n; j++) {
23             if (arr[i] == arr[j]) return true;
24         }
25     }
26     return false;
27 }
28
29 // O(log n) - Logarítmico
30 int busquedaBinaria(vector<int>& arr, int objetivo) {
31     int izq = 0;
32     int der = arr.size() - 1;
33     while (izq <= der) {
34         int medio = izq + (der - izq) / 2;
35         if (arr[medio] == objetivo) return medio;
36         if (arr[medio] < objetivo) izq = medio + 1;
37         else der = medio - 1;
38     }
39     return -1;
40 }

```

Complejidad Big-O

Notación & Nombre & Ejemplo real & ¿Usable?
$O(1)$ & Constante & Array access, hash lookup & Ideal
$O(\log n)$ & Logarítmico & Binary search, BST ops & Muy bueno
$O(n)$ & Lineal & Loop simple, linear search & Aceptable
$O(n \log n)$ & Linealítmico & Merge sort, Heap sort & Bueno
$O(n^2)$ & Cuadrático & Bubble sort, nested loops & Malo
$O(2^n)$ & Exponencial & Subsets, recursion fatal & Terrible

En entrevistas

- Big-O ignora constantes y términos menores: $O(3n + 5) \rightarrow O(n)$.
- Siempre menciona tiempo Y espacio.
- **Regla de oro:** Si ves que tu solución tiene $O(n^2)$ y el problema involucra búsqueda, piensa si un hash map te da $O(n)$ en su lugar.

 Resumen rápido

- Big-O = qué tan rápido crece el tiempo al crecer el input.
- Siempre describe el PEOR caso.
- $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$.
- Hash maps convierten muchos $O(n^2)$ en $O(n)$.

Capítulo 2

Complejidad Espacial (Space Complexity)

¿Qué es?

Imagina que tu programa es una receta de cocina. Big-O en tiempo mide cuánto TARDAS en cocinar. Complejidad espacial mide cuántas OLLAS Y SARTENES necesitas mientras cocinas.

Si para preparar una ensalada de 10 ingredientes necesitas una tabla de picar pequeña, pero para 100 ingredientes necesitas una mesa gigante, eso significa que el espacio crece con el número de ingredientes.

En programación, la complejidad espacial mide cuánta MEMORIA EXTRA (RAM) utiliza tu algoritmo para resolver un problema a medida que el input se hace más grande.

Dicho de otra forma

La Space Complexity analiza el uso de recursos de memoria. Es crucial distinguir entre:

1. **Input Space:** La memoria para guardar los datos de entrada.
2. **Auxiliary Space:** La memoria EXTRA o temporal que el algoritmo crea.

En la mayoría de las entrevistas, cuando preguntan "Space Complexity", se refieren al Auxiliary Space. Si tu algoritmo modifica el input original sin crear estructuras nuevas, se dice que es "In-place" y su complejidad espacial es $O(1)$.

DIAGRAMA

Cuando usas recursión, cada llamada ocupa un "piso" en el stack:

```
factorial(4)
  |-- factorial(3)
    |-- factorial(2)
      |-- factorial(1)  <-- Tope del stack
```

Cada nivel de recursión consume memoria adicional en el Call Stack.

⚙️ ¿Cómo funciona?

$O(1)$ — Espacio Constante Solo usas unas cuantas variables fijas sin importar el input. Ejemplo: Hacer un swap de dos números usando una variable temporal.

$O(n)$ — Espacio Lineal Creas una estructura (como un array o vector) proporcional al input. Ejemplo: Copiar una lista de n elementos a una nueva lista.

$O(n^2)$ — Espacio Cuadrático Creas una estructura bidimensional (matriz) de $n \times n$. Ejemplo: Una tabla de adyacencia para un grafo denso.

📊 Complejidad Big-O

Algoritmo & Espacio & Razón

Acceso a Array & $O(1)$ & No requiere memoria extra

Búsqueda Lineal & $O(1)$ & Solo usa un par de índices

Búsqueda Binaria iter. & $O(1)$ & Solo usa izq/der/mid

Búsqueda Binaria recur. & $O(\log n)$ & $\log(n)$ llamadas en el stack

Merge Sort & $O(n)$ & Necesita un array auxiliar

Quick Sort & $O(\log n)$ & Promedio de llamadas stack

Crear matriz $n \times n$ & $O(n^2)$ & Espacio bidimensional

🎯 En entrevistas

- En Google preguntan SIEMPRE espacio Y tiempo. No esperes a que pregunten.
- **"In-place algorithm"**: Significa que el algoritmo tiene $O(1)$ espacio auxiliar porque modifica el input original directamente.
- **Trade-off clásico**: A veces puedes hacer que un programa sea más rápido (tiempo) si estás dispuesto a usar más memoria (espacio). Ejemplo: Memoization en Dynamic Programming.

📋 Resumen rápido

- Space Complexity = cuánta RAM EXTRA usa el algoritmo.
- No cuenta el espacio del input original (Auxiliary Space).
- Estructuras de datos (arrays, maps) creadas $\rightarrow O(\text{tamaño})$.
- Recursión $\rightarrow$ cada nivel cuenta como $O(1)$ en el Call Stack.
- $O(1)$ espacio = "In-place", es lo más eficiente en memoria.

Estructuras Lineales

Capítulo 3

Arrays (Vectores)

¿Qué es?

Los vagones de un tren numerados en orden. El vagón 0, el vagón 1, el vagón 2... Cada vagón tiene exactamente UN pasajero (dato).

Para subir a cualquier vagón, solo dices el número (el índice) y ¡listo! No tienes que pasar por todos los vagones anteriores. Esto es posible porque todos los vagones están pegaditos uno tras otro en la vía.

Un array es una colección de elementos del mismo tipo guardados en posiciones de memoria CONTIGUAS (una tras otra).

```
1 El Array es la estructura de datos más básica y fundamental. Su
2 característica principal es el acceso aleatorio (Random Access) en
3 tiempo constante  $O(1)$ .
4
5 En lenguajes modernos como C++, solemos usar "Dynamic Arrays"
6 (como std::vector), que pueden crecer automáticamente si se
7 llenan,
8 aunque por debajo siguen siendo un bloque de memoria contigua.
```

Diagrama

```
Índice:  [0] [1] [2] [3] [4]
Valor:   [ 5] [ 2] [ 8] [ 1] [ 9]
          ↑
        Dirección de memoria contigua →→→→→→→→→→

Insertar 99 en el índice 2:
ANTES:   [ 5] [ 2] [ 8] [ 1] [ 9]
          ↓  ↓  ← desplazar →
DESPUÉS: [ 5] [ 2] [99] [ 8] [ 1] [ 9]  ←  $O(n)$  porque movimos 8, 1 y 9
```

```
1 1. Acceso: Vas directo al índice `i`. Es instantáneo.
2 2. Búsqueda: Tienes que revisar uno por uno hasta encontrar el
3   valor.
4 3. Inserción:
5   - Al final: Si hay espacio, es instantáneo.
6   - Al inicio/medio: Tienes que "empujar" a todos los de la
   derecha
   un lugar para hacer espacio.
```

4. Eliminación:

- Al final: Borrás y ya.
- Al inicio/medio: Tienes que "jalar" a todos los de la derecha un lugar a la izquierda para cerrar el hueco.

PSEUDOCÓDIGO

// Insertar en medio

función insertar(array, valor, posicion):

PARA i desde tamaño hasta posicion + 1:

array[i] = array[i-1] *// desplazar a la derecha*

array[posicion] = valor

// Eliminar en medio

función eliminar(array, posicion):

PARA i desde posicion hasta tamaño - 2:

array[i] = array[i+1] *// desplazar a la izquierda*

CÓDIGO C++

#include <iostream>

#include <vector> *// En C++ usamos vector para arrays dinámicos*

#include <algorithm>

using namespace std;

int main() {

// 1. Declaración e inicialización

vector<int> arr = {5, 2, 8, 1, 9};

*// 2. Acceso por índice -> O(1)*cout << "Elemento en idx 2: " << arr[2] << endl; *// 8**// 3. Insertar al final -> O(1) amortizado*arr.push_back(10); *// [5, 2, 8, 1, 9, 10]**// 4. Insertar en medio -> O(n)**// Insertamos el 99 en la posición 2*arr.insert(arr.begin() + 2, 99); *// [5, 2, 99, 8, 1, 9, 10]**// 5. Eliminar en medio -> O(n)**// Borramos el elemento en la posición 4*arr.erase(arr.begin() + 4); *// [5, 2, 99, 8, 9, 10]**// 6. Tamaño del array*

cout << "Tamaño: " << arr.size() << endl;

// 7. Recorrer el array -> O(n)

for(int x : arr) {

cout << x << " ";

```

58     }
59     cout << endl;
60
61     return 0;
62 }
63
64 COMPLEJIDAD (Big-O)
65
66
67 Operación          Tiempo          Espacio
68
69 Access             O(1)             O(1)
70 Search unsorted    O(n)             O(1)
71 Search sorted      O(log n)         O(1)
72 Insert end         O(1)*            O(1)
73 Insert middle      O(n)             O(1)
74 Delete end         O(1)             O(1)
75 Delete middle      O(n)             O(1)
76 *amortizado en dynamic array (vector)

```

En entrevistas

Arrays son la estructura de datos más común en entrevistas. Domínalos.

Patrones clave: - "Two Pointers": Usar un índice al inicio y otro al final. - "Sliding Window": Una ventana que se desliza para encontrar subarrays. - "Prefix Sum": Pre-calcular sumas para responder rangos en $O(1)$.

Off-by-one errors: El error más común es salirte del rango (acceder a `arr[n]` cuando el tamaño es `n`). Siempre usa `i < n`.

Pregunta clave: "¿El array está ordenado?". Si está ordenado, casi siempre la respuesta involucra Binary Search $O(\log n)$.

- 1 •
- 2 Array = bloque de memoria contigua. •
- 3 Acceso instantáneo $O(1)$ si conoces el índice. •
- 4 Insertar/Eliminar en medio es LENTO $O(n)$ por los desplazamientos. •
- 5 En C++: `std::vector` es el rey. •
- 6 Muy eficiente en espacio (mínimo overhead). •
- 7 Ideal para cuando conoces el tamaño o solo agregas/quitas al final

Capítulo 4

Linked Lists (Listas Enlazadas)

¿Qué es?

Una cacería del tesoro . Cada pista (node) te dice dónde está la SIGUIENTE pista.

No sabes dónde está la pista #5 sin seguir la cadena desde el inicio: pista 1 → pista 2 → pista 3 → pista 4 → pista 5.

A diferencia de los arrays (vagones pegaditos), los nodos de una Linked List pueden estar en cualquier parte de la memoria. Lo único que los mantiene unidos es que cada uno sabe la dirección (pointer) del siguiente.

Dicho de otra forma

Una Linked List es una estructura lineal donde los elementos no se guardan en posiciones contiguas de memoria. Cada elemento es un objeto llamado "Node", que contiene: 1. El valor (data). 2. Un puntero (next) al siguiente nodo.

Esto permite insertar y eliminar elementos en los extremos en $O(1)$ sin tener que desplazar a nadie, pero sacrificas el acceso aleatorio (no puedes ir directo al índice 'i').

Diagrama

Singly Linked List:

HEAD

↓

[3|•] → [7|•] → [1|•] → [9|NULL]
node0 node1 node2 node3

Doubly Linked List:

HEAD

TAIL

↓

↓

[NULL|3|•] [•|7|•] [•|1|•] [•|9|NULL]

Insertar 5 después del node1 (valor 7):

ANTES: [3] → [7] → [1] → [9]

↓

DESPUÉS: [3] → [7] → [5] → [1] → [9] ← $O(1)$ si ya tienes el puntero

1. Acceso/Búsqueda: Tienes que empezar en el HEAD e ir saltando de

```

2     nodo en nodo.  $O(n)$ .
3 2. Inserción:
4     - Al Head: Creas nodo, apuntas su `next` al viejo head,
      actualizas
5     head.  $O(1)$ .
6     - En medio: Llegas a la posición, ajustas los punteros para
      incluir
7     el nuevo nodo.
8 3. Eliminación:
9     - Del Head: Mueves el head al `head->next`.  $O(1)$ .
10    - En medio: Ajustas el puntero del nodo anterior para que "salte
      "
11    al nodo que quieres borrar.

```

PSEUDOCÓDIGO

```

16 // Insertar al inicio
17 función insertarAlInicio(valor):
18     nuevoNodo = crear Nodo(valor)
19     nuevoNodo.next = head
20     head = nuevoNodo
21
22 // Buscar valor
23 función buscar(valor):
24     actual = head
25     MIENTRAS actual != NULL:
26         SI actual.valor == valor: return VERDADERO
27         actual = actual.next
28     return FALSO

```

CÓDIGO C++

```

33 #include <iostream>
34 using namespace std;
35
36 // Definición del Nodo
37 struct Node {
38     int data;
39     Node* next;
40     Node(int val) : data(val), next(nullptr) {} // Constructor
41 };
42
43 class LinkedList {
44 public:
45     Node* head;
46     LinkedList() : head(nullptr) {}
47
48     // Insertar al inicio -> O(1)
49     void insertHead(int val) {

```

```
50     Node* newNode = new Node(val);
51     newNode->next = head;
52     head = newNode;
53 }
54
55 // Insertar al final -> O(n) (sin tail pointer)
56 void insertTail(int val) {
57     Node* newNode = new Node(val);
58     if (!head) {
59         head = newNode;
60         return;
61     }
62     Node* temp = head;
63     while (temp->next) temp = temp->next; // Recorrer hasta el
        final
64     temp->next = newNode;
65 }
66
67 // Buscar -> O(n)
68 bool search(int val) {
69     Node* temp = head;
70     while (temp) {
71         if (temp->data == val) return true;
72         temp = temp->next;
73     }
74     return false;
75 }
76
77 // Imprimir lista
78 void print() {
79     Node* temp = head;
80     while (temp) {
81         cout << temp->data << " -> ";
82         temp = temp->next;
83     }
84     cout << "NULL" << endl;
85 }
86 };
87
88 int main() {
89     LinkedList list;
90     list.insertHead(10);
91     list.insertHead(20);
92     list.insertTail(5);
93
94     list.print(); // 20 -> 10 -> 5 -> NULL
95
96     cout << "¿Está el 10? " << (list.search(10) ? "Sí" : "No") <<
        endl;
97
98     return 0;
```

```

99 }
100
101 COMPLEJIDAD (Big-O)
102
103
104 Operación          Tiempo    vs Array
105
106 Access por índice  O(n)      Array: O(1) ← array gana
107 Search             O(n)      Igual
108 Insert head        O(1)      Array: O(n) ← linked list gana
109 Insert tail        O(1)*     Array: O(1) amort. → Igual
110 Insert medio       O(n)      Igual (pero no desplaza datos)
111 Delete head        O(1)      Array: O(n) ← linked list gana
112 *con tail pointer

```

En entrevistas

Patrón "Fast & Slow Pointers" (Tortoise and Hare): - Sirve para detectar ciclos (si se encuentran, hay ciclo). - Sirve para encontrar el medio (cuando fast llega al final, slow está en el medio).

Dummy Node: Crear un nodo falso al inicio ayuda a simplificar casos borde (como insertar en una lista vacía o borrar el head).

Memory Management: En C++, recuerda que cada 'new' necesita un 'delete' para evitar memory leaks (fugas de memoria).

Pregunta clave: "¿Es singly o doubly?". Si es doubly, puedes ir hacia atrás, lo que facilita borrar un nodo si ya tienes el puntero.

Resumen rápido

- Cadena de nodos dispersos en memoria.
- Acceso secuencial (no puedes saltar).
- Excelente para insertar/eliminar al inicio $O(1)$.
- No desperdicia memoria (solo usa lo que necesita).
- Invertir una Linked List es un problema clásico (usar 3 punteros).
- Usa más memoria por elemento que un array (por el puntero next).

Capítulo 5

Fast & Slow Pointers (Tortuga y Liebre)

¿Qué es?

Dos corredores en una pista circular : uno lento (tortuga, avanza 1 paso) y uno rápido (liebre, avanza 2 pasos). Si la pista tiene un bucle, tarde o temprano el rápido le da la vuelta y ALCANZA al lento por detrás. Si la pista termina (hay una meta), el rápido llega al final y nunca se encuentran.

Ese simple truco resuelve un montón de problemas de listas enlazadas: detectar ciclos, encontrar el nodo del medio y hallar dónde empieza el bucle, todo sin usar memoria extra.

Dicho de otra forma

La técnica Fast & Slow (algoritmo de la tortuga y la liebre de Floyd) usa dos punteros que recorren la lista a velocidades distintas: uno avanza de 1 en 1 y otro de 2 en 2.

Sus tres usos clásicos:

- **Detectar ciclo:** si en algún momento `fast == slow`, hay ciclo. Si `fast` llega a NULL, no lo hay.
- **Encontrar el medio:** cuando `fast` llega al final, `slow` está justo a la mitad.
- **Inicio del ciclo:** tras el encuentro, mueve un puntero al `head` y avanza ambos de 1 en 1; se cruzan en el inicio del bucle.

Todo en $O(n)$ tiempo y $O(1)$ espacio: ahí está su valor.

Diagrama

Lista con ciclo:

```
1 -> 2 -> 3 -> 4 -> 5
                ^
```

```
slow (1 paso):  1  2  3  4  5  3  4  ...
fast (2 pasos): 1  3  5  4  3  5  4  ...
```

encuentro en el nodo 4 => HAY CICLO

Encontrar el medio (lista sin ciclo):

1 -> 2 -> 3 -> 4 -> 5 -> NULL

fast llega a NULL => slow queda en 3 (el medio)

```

1 DETECTAR CICLO
2 - slow = head, fast = head.
3 - Avanza slow 1 y fast 2 mientras fast y fast->next existan.
4 - Si slow == fast en algún punto -> hay ciclo.
5
6 INICIO DEL CICLO (después del encuentro)
7 - Deja un puntero en el punto de encuentro, mueve otro al head.
8 - Avanza ambos de 1 en 1; donde se crucen es el inicio del ciclo.
9
10 PSEUDOCÓDIGO (detectar + inicio de ciclo)
11
12
13 función detectarCiclo(head):
14     slow = head; fast = head
15     MIENTRAS fast Y fast.next:
16         slow = slow.next
17         fast = fast.next.next
18         SI slow == fast:                                // hay ciclo
19             p = head
20             MIENTRAS p != slow:
21                 p = p.next; slow = slow.next
22             return p                                    // inicio del ciclo
23     return NULL                                         // no hay ciclo
24
25 CÓDIGO C++
26
27
28 struct Node { int val; Node* next; };
29
30 // ¿Tiene ciclo?
31 bool hasCycle(Node* head) {
32     Node *slow = head, *fast = head;
33     while (fast && fast->next) {
34         slow = slow->next;
35         fast = fast->next->next;
36         if (slow == fast) return true;    // se alcanzaron
37     }
38     return false;                            // fast llegó a NULL
39 }
40
41 // Nodo del medio
42 Node* middleNode(Node* head) {
43     Node *slow = head, *fast = head;
44     while (fast && fast->next) {

```

```

45     slow = slow->next;
46     fast = fast->next->next;
47 }
48 return slow;                                // mitad exacta (o 2da
        mitad si par)
49 }
50
51 COMPLEJIDAD (Big-O)
52
53
54 Problema            Tiempo    Espacio
55
56 Detectar ciclo      O(n)      O(1)
57 Encontrar medio    O(n)      O(1)
58 Inicio del ciclo   O(n)      O(1)
59
60 * La gran ventaja es el O(1) de memoria (sin HashSet de nodos
    vistos).

```

```

1
2 Otros usos del patrón:
3 - Palíndromo de lista: encuentra el medio, invierte la 2da mitad
  y
4   compara.
5 - Happy Number: aplica fast/slow sobre la secuencia de sumas de
6   cuadrados de dígitos para detectar el ciclo.
7 - Eliminar el N-ésimo desde el final: separa dos punteros por N
8   nodos y avánzalos juntos.

```

🔗 En entrevistas

”Linked List Cycle” (LeetCode 141/142): 141 pide detectar, 142 pide el nodo donde empieza. Ambos son Floyd puro.

¿Por qué funciona lo del inicio? La distancia del head al inicio del ciclo es igual a la distancia del punto de encuentro al inicio del ciclo (avanzando en el bucle). Sabértelo justifica el algoritmo.

Cuida los NULL: siempre valida `fast != NULL` Y `fast->next != NULL` antes de hacer `fast->next->next`, o provocas un segfault.

Alternativa con HashSet: guardar nodos visitados también detecta ciclos, pero usa $O(n)$ memoria. Floyd es superior por su $O(1)$.

📄 Resumen rápido

- Dos punteros: lento (1 paso) y rápido (2 pasos).
- Se encuentran = hay ciclo; fast llega a NULL = no hay.
- fast al final -> slow queda en el medio.
- Inicio del ciclo: un puntero al head, avanzar ambos de 1 en 1.
- $O(n)$ tiempo, $O(1)$ espacio (mejor que HashSet).
- Clásicos: cycle detection, medio, palíndromo, happy number.

Capítulo 6

Stacks (Pilas)

¿Qué es?

Una pila de platos en el fregadero. Solo puedes poner platos encima (push) y solo puedes agarrar el de hasta arriba (pop).

Si quieres el plato de abajo, debes quitar todos los de encima primero. Es una estructura "LIFO" (Last In, First Out): el último en entrar es el primero en salir.

Imagina un tubo de papas Pringles : la última papa que metieron en la fábrica es la primera que te comes al abrir el tubo.

Dicho de otra forma

Un Stack es una estructura de datos lineal de tipo ADT (Abstract Data Type) que restringe el acceso a un solo extremo: el "Top".

Es fundamental para el funcionamiento de las computadoras, ya que el "Call Stack" de los lenguajes de programación usa esta lógica para gestionar las llamadas a funciones y la recursión.

Diagrama

```
push(A) → push(B) → push(C)
```

```
[C] ← TOP (C es el último en entrar)
[B]
[A] ← BOTTOM
```

```
pop() → devuelve C y lo quita, queda:
```

```
[B] ← TOP (ahora B es el nuevo tope)
[A]
```

1. `push(x)`: Agregas el elemento `x` al tope de la pila. $O(1)$.
2. `pop()`: Quitas el elemento del tope. $O(1)$.
3. `peek()` / `top()`: Miras qué hay en el tope sin quitarlo. $O(1)$.
4. `isEmpty()`: Verificas si la pila está vacía. $O(1)$.
5. `size()`: Devuelves cuántos elementos hay. $O(1)$.

PSEUDOCÓDIGO

```
9
10 // Validar paréntesis balanceados usando un Stack
11 función esValido(cadena):
12     pila = crear Stack
13     PARA cada caracter EN cadena:
14         SI caracter == '(':
15             pila.push('(')
16         SINO SI caracter == ')':
17             SI pila.isEmpty(): return FALSO
18             pila.pop()
19     return pila.isEmpty()
20
21 CÓDIGO C++
22
23
24 #include <iostream>
25 #include <stack>      // STL Stack
26 #include <vector>
27 using namespace std;
28
29 // Implementación manual simple usando un vector
30 class MyStack {
31 private:
32     vector<int> data;
33 public:
34     void push(int x) { data.push_back(x); }
35     void pop() { if(!data.empty()) data.pop_back(); }
36     int top() { return data.back(); }
37     bool isEmpty() { return data.empty(); }
38 };
39
40 int main() {
41     // 1. Usando STL Stack
42     stack<int> s;
43
44     s.push(10);
45     s.push(20);
46     s.push(30);
47
48     cout << "Tope actual: " << s.top() << endl; // 30
49
50     s.pop(); // Quita el 30
51     cout << "Nuevo tope: " << s.top() << endl; // 20
52
53     // 2. Recorrer (vacizando)
54     while(!s.empty()) {
55         cout << s.top() << " ";
56         s.pop();
57     }
58     cout << endl;
59 }
```

```

60     return 0;
61 }
62
63 COMPLEJIDAD (Big-O)
64
65
66 Operación          Tiempo    Espacio
67
68 push(x)             O(1)      O(1)
69 pop()               O(1)      O(1)
70 peek() / top()      O(1)      O(1)
71 isEmpty()           O(1)      O(1)
72 Total (n datos)     -          O(n)

```

En entrevistas

Problema de Oro: "Valid Parentheses". Si ves paréntesis, corchetes o llaves en un problema, ¡usa un stack!

Recursión vs Stack: Cualquier problema recursivo se puede resolver usando un Stack de forma iterativa. A veces esto evita el Stack Overflow.

Stack Underflow: Intentar hacer 'pop()' o 'top()' en una pila vacía. SIEMPRE verifica '!s.empty()' antes de acceder al tope.

Patrón "Monotonic Stack": Un stack donde los elementos siempre están en orden (creciente o decreciente). Clave para problemas como "Next Greater Element" o "Daily Temperatures".

Resumen rápido

- LIFO: Last In, First Out.
- Todas las operaciones básicas son $O(1)$ (instantáneas).
- Usos: Historial del navegador (Undo/Redo), procesar expresiones matemáticas, DFS (Depth First Search).
- Muy eficiente pero limitado: solo ves el tope.
- La base de la recursión en cualquier lenguaje.

Capítulo 7

Queues (Filas / Colas)

¿Qué es?

La fila para entrar al cine o al banco. El primero que llega es el primero en ser atendido. No puedes "colarte" al frente; llegas al final y esperas tu turno.

Es una estructura "FIFO" (First In, First Out): el primero que entra es el primero en salir. Es lo opuesto al Stack.

Dicho de otra forma

Una Queue es una estructura lineal que restringe el acceso: - Solo puedes AGREGAR (Enqueue) por el final (Back/Rear). - Solo puedes QUITAR (Dequeue) por el frente (Front).

Existen variaciones importantes: 1. Deque: Puedes insertar/eliminar por ambos extremos. 2. Priority Queue: Los elementos salen según su importancia, no solo por orden de llegada (implementada usualmente con un Heap).

Diagrama

`enqueue(A) → enqueue(B) → enqueue(C)`

FRONT		BACK
↓		↓
[A]	→ [B]	→ [C]

`dequeue()` → devuelve A y lo quita, queda:

FRONT	BACK
↓	↓
[B]	→ [C]

1. `enqueue(x)`: Agregas el elemento `x` al final de la fila. $O(1)$.
2. `dequeue()`: Quitas el primer elemento del frente. $O(1)$.
3. `front()` / `peek()`: Miras quién es el primero sin quitarlo. $O(1)$.
4. `isEmpty()`: Verificas si hay alguien en la fila. $O(1)$.
5. `size()`: Devuelves cuántas personas hay esperando. $O(1)$.

PSEUDOCÓDIGO

```

10 // BFS (Breadth First Search) básico usando una Queue
11 función BFS(inicio):
12     fila = crear Queue
13     fila.enqueue(inicio)
14     marcar inicio como visitado
15
16     MIENTRAS fila NO esté vacía:
17         actual = fila.dequeue()
18         PARA cada vecino de actual:
19             SI vecino NO visitado:
20                 fila.enqueue(vecino)
21                 marcar vecino como visitado
22
23 CÓDIGO C++
24
25
26 #include <iostream>
27 #include <queue>      // STL Queue
28 #include <deque>      // STL Deque (Double-ended queue)
29 using namespace std;
30
31 int main() {
32     // 1. Queue estándar (FIFO)
33     queue<int> q;
34
35     q.push(10); // Enqueue
36     q.push(20);
37     q.push(30);
38
39     cout << "Primero en la fila: " << q.front() << endl; // 10
40     cout << "Último en la fila: " << q.back() << endl;   // 30
41
42     q.pop(); // Dequeue (quita el 10)
43     cout << "Nuevo primero: " << q.front() << endl;      // 20
44
45     // 2. Deque (Doble extremo)
46     deque<int> dq;
47     dq.push_back(100);
48     dq.push_front(50); // [50, 100]
49     dq.pop_back();     // [50]
50
51     cout << "Frente de deque: " << dq.front() << endl;
52
53     return 0;
54 }
55
56 COMPLEJIDAD (Big-O)
57
58
59 Operación          Tiempo          Espacio
60

```

61	<code>enqueue(x)</code>	$O(1)$	$O(1)$
62	<code>dequeue()</code>	$O(1)$	$O(1)$
63	<code>front()</code> / <code>peek()</code>	$O(1)$	$O(1)$
64	<code>isEmpty()</code>	$O(1)$	$O(1)$
65	Total (n datos)	-	$O(n)$



En entrevistas

Sin Queue no hay BFS: Si el problema pide "camino más corto" en un grafo sin pesos o explorar un árbol por niveles, vas a usar una Queue.

Priority Queue: Úsala cuando necesites el "máximo" o "mínimo" elemento de forma constante mientras insertas nuevos datos.

Implementación con Array: Cuidado, si implementas una Queue con un Array simple, el 'dequeue' podría ser $O(n)$ si tienes que mover todos los elementos. Por eso se usan "Circular Queues".

"Sliding Window Maximum": Un problema clásico de Google que se resuelve de forma óptima usando un Deque.



Resumen rápido

- FIFO: First In, First Out.
- Operaciones $O(1)$ en el frente y el final.
- Ideal para: Procesamiento de tareas, buffers, algoritmos de grafos como BFS.
- Deque: Permite meter/sacar por ambos lados (muy flexible).
- Priority Queue: Saca al que tiene más "rango", no al que llegó primero.

Estructuras No Lineales

Capítulo 8

🌳 Binary Search Trees (Árboles Binarios de Búsqueda)

🤖 ¿Qué es?

Un árbol genealógico de cabeza. La raíz (root) está hasta arriba. Cada persona (nodo) puede tener máximo dos hijos.

Pero es un árbol inteligente con una regla de oro: - Todo lo que está a la IZQUIERDA de un nodo es MENOR. - Todo lo que está a la DERECHA de un nodo es MAYOR.

Es como una biblioteca donde los libros están tan bien ordenados que puedes encontrar cualquiera en segundos, sin revisar todos los estantes.

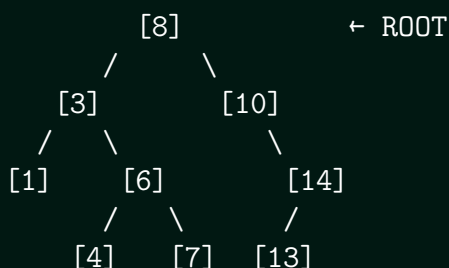
💬 Dicho de otra forma

Un Binary Search Tree (BST) es una estructura de datos no lineal formada por nodos. Cada nodo contiene un valor y punteros a sus hijos izquierdo y derecho.

La propiedad de orden ($\text{izq} < \text{raíz} < \text{der}$) permite que las operaciones de búsqueda, inserción y eliminación se realicen en tiempo logarítmico $O(\log n)$, siempre y cuando el árbol esté balanceado.

🤖 Diagrama

BST de ejemplo:



- Leaf Nodes (hojas): 1, 4, 7, 13 (no tienen hijos).
- Height (altura): 3 (niveles desde la raíz).

1. Search: Comparas con la raíz. ¿Es menor? Vas a la izquierda.
2. ¿Es mayor? Vas a la derecha. Repites hasta encontrarlo o llegar a un callejón sin salida (NULL).
- 3.

- 4 2. Insert: Buscas el lugar donde debería estar el valor siguiendo la
- 5 regla de orden y lo agregas como una nueva hoja.
- 6 3. Delete: Es la más compleja. Tiene 3 casos:
- 7 - Nodo es una hoja: Solo lo borras.
- 8 - Nodo tiene 1 hijo: El hijo sube a ocupar el lugar del padre.
- 9 - Nodo tiene 2 hijos: Buscas al sucesor (el más pequeño del lado
- 10 derecho) para que ocupe su lugar.
- 11 4. Traversal (Recorrido):
- 12 - Inorder: Izquierda -> Raíz -> Derecha (¡Te da los números en
- 13 orden!).

14 PSEUDOCÓDIGO

```
15
16
17 // Búsqueda recursiva en un BST
18 función buscar(nodo, valor):
19     SI nodo es NULL: return FALSO
20     SI nodo.valor == valor: return VERDADERO
21     SI valor < nodo.valor:
22         return buscar(nodo.izq, valor)
23     SINO:
24         return buscar(nodo.der, valor)
25
```

26 CÓDIGO C++

```
27
28
29 #include <iostream>
30 using namespace std;
31
32 struct Node {
33     int data;
34     Node *left, *right;
35     Node(int val) : data(val), left(nullptr), right(nullptr) {}
36 };
37
38 class BST {
39 public:
40     Node* root;
41     BST() : root(nullptr) {}
42
43     // Insertar -> O(log n) promedio
44     Node* insert(Node* node, int val) {
45         if (!node) return new Node(val);
46         if (val < node->data)
47             node->left = insert(node->left, val);
48         else
49             node->right = insert(node->right, val);
50         return node;
51     }
52
```

```

53 // Búsqueda -> O(log n) promedio
54 bool search(Node* node, int val) {
55     if (!node) return false;
56     if (node->data == val) return true;
57     if (val < node->data) return search(node->left, val);
58     return search(node->right, val);
59 }
60
61 // Inorder Traversal -> O(n)
62 void inorder(Node* node) {
63     if (!node) return;
64     inorder(node->left);
65     cout << node->data << " ";
66     inorder(node->right);
67 }
68 };
69
70 int main() {
71     BST tree;
72     tree.root = tree.insert(tree.root, 8);
73     tree.insert(tree.root, 3);
74     tree.insert(tree.root, 10);
75     tree.insert(tree.root, 1);
76     tree.insert(tree.root, 6);
77
78     cout << "Inorder: ";
79     tree.inorder(tree.root); // 1 3 6 8 10
80     cout << endl;
81
82     cout << "¿Está el 6? " << (tree.search(tree.root, 6) ? "Sí" : "
83         No") << endl;
84
85     return 0;
86 }

```

COMPLEJIDAD (Big-O)

Operación	Promedio	Peor caso (Degenerado)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Traversal	$O(n)$	$O(n)$
Espacio	$O(n)$	$O(n)$

* Peor caso ocurre cuando el árbol se vuelve una línea (como linked list).

En entrevistas

"Inorder Traversal = Sorted Array": Si recorres un BST en modo Inorder, siempre obtendrás los elementos ordenados de menor a mayor.

Lowest Common Ancestor (LCA): Un problema de Google súper común. En un BST es fácil: el LCA es el primer nodo cuyo valor está ENTRE los dos valores que buscas.

Balanceo: Un BST normal puede volverse muy ineficiente ($O(n)$) si insertas los números ya ordenados. Para evitarlo existen los Self-balancing Trees (como AVL).

Pregunta clave: "¿Es un Binary Tree o un Binary Search Tree?". El segundo tiene la propiedad de orden, el primero no.

Resumen rápido

- Izquierda menor, Derecha mayor.
- Búsqueda rápida $O(\log n)$ como adivinar un número.
- Inorder traversal = lista ordenada.
- El peor caso es $O(n)$ si el árbol no está balanceado.
- Términos: Root (cima), Leaf (hoja), Parent (padre), Child (hijo).
- Altura del árbol = longitud del camino más largo a una hoja.

Capítulo 9

AVL Trees (Árboles Binarios Balanceados)

¿Qué es?

Un móvil de bebé . Si pones mucho peso de un lado, el móvil se inclina y queda chueco. Para que funcione bien, debes mover las piezas hasta que quede perfectamente equilibrado.

Un árbol AVL es un Binary Search Tree (BST) que tiene una "manía" por el orden: no permite que una rama sea mucho más larga que otra. Si al insertar un número el árbol se desbalancea, el árbol se "tuerce" a sí mismo (rotación) para volver al equilibrio.

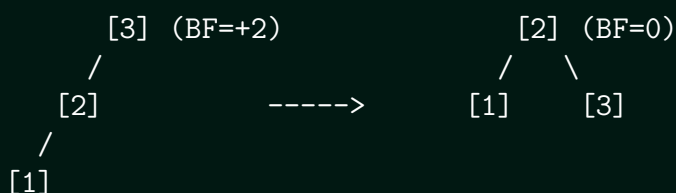
Dicho de otra forma

El AVL es el primer árbol auto-balanceable de la historia. Su regla es estricta: para cada nodo, la diferencia de altura entre su hijo izquierdo y derecho (llamada Balance Factor) solo puede ser -1, 0 o 1.

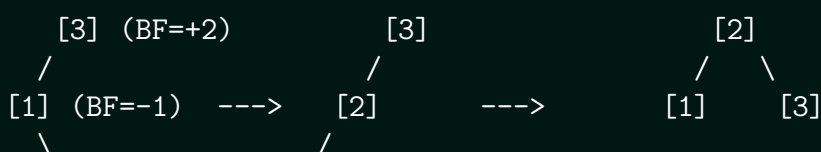
Si la diferencia es 2 o -2, el árbol realiza rotaciones (Simples o Dobles) para restaurar el balance. Esto garantiza que la altura del árbol sea siempre $O(\log n)$, evitando el peor caso de los BST normales.

Diagrama

Rotación Simple a la Derecha (LL Rotation):



Rotación Doble (LR Rotation):



[2]

[1]

- 1 1. Insert: Igual que un BST, pero después de insertar, el árbol
revisa
- 2 la altura de los nodos hacia arriba. Si encuentra uno
desbalanceado,
- 3 aplica una de las 4 rotaciones (LL, RR, LR, RL).
- 4 2. Delete: También igual que BST, pero requiere revisar el balance
hacia arriba, pudiendo causar múltiples rotaciones.
- 5
- 6 3. Search: Exactamente igual que en un BST, pero con la garantía de
que siempre será rápido ($O(\log n)$).
- 7

PSEUDOCÓDIGO

```

12 // Obtener factor de balance
13 función obtenerBalance(nodo):
14     SI nodo es NULL: return 0
15     return altura(nodo.izq) - altura(nodo.der)
16
17 // Rotación a la derecha
18 función rotarDerecha(y):
19     x = y.izq
20     T2 = x.der
21     x.der = y
22     y.izq = T2
23     actualizarAlturas(y, x)
24     return x

```

CÓDIGO C++

```

29 #include <iostream>
30 #include <algorithm>
31 using namespace std;
32
33 struct Node {
34     int key, height;
35     Node *left, *right;
36     Node(int k) : key(k), height(1), left(nullptr), right(nullptr)
37     {}
38 };
39
40 int getHeight(Node* n) { return n ? n->height : 0; }
41
42 Node* rightRotate(Node* y) {
43     Node* x = y->left;
44     Node* T2 = x->right;
45     x->right = y;
46     y->left = T2;
47     y->height = max(getHeight(y->left), getHeight(y->right)) + 1;

```

```

47     x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
48     return x;
49 }
50
51 Node* leftRotate(Node* x) {
52     Node* y = x->right;
53     Node* T2 = y->left;
54     y->left = x;
55     x->right = T2;
56     x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
57     y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
58     return y;
59 }
60
61 Node* insert(Node* node, int key) {
62     if (!node) return new Node(key);
63     if (key < node->key) node->left = insert(node->left, key);
64     else if (key > node->key) node->right = insert(node->right, key
65 );
66     else return node;
67
68     node->height = 1 + max(getHeight(node->left), getHeight(node->
69 right));
70     int balance = getHeight(node->left) - getHeight(node->right);
71
72     if (balance > 1 && key < node->left->key) return rightRotate(
73 node);
74     if (balance < -1 && key > node->right->key) return leftRotate(
75 node);
76     if (balance > 1 && key > node->left->key) {
77         node->left = leftRotate(node->left);
78         return rightRotate(node);
79     }
80     if (balance < -1 && key < node->right->key) {
81         node->right = rightRotate(node->right);
82         return leftRotate(node);
83     }
84     return node;
85 }
86
87 int main() {
88     Node* root = nullptr;
89     root = insert(root, 10);
90     root = insert(root, 20);
91     root = insert(root, 30); // Esto causará una rotación RR
92
93     cout << "Raíz del árbol balanceado: " << root->key << endl; //
94         Debería ser 20
95     return 0;
96 }

```

```

93 COMPLEJIDAD (Big-O)
94
95
96 Operación      Tiempo Garantizado
97
98 Search         O(log n)
99 Insert         O(log n)
100 Delete         O(log n)
101 Espacio        O(n)
102
103 * A diferencia del BST normal, el peor caso en AVL sigue siendo O(
    log n).

```

```

1
2 ¿Por qué AVL y no BST?: La respuesta siempre es "Balance
   Garantizado".
3   Un AVL nunca se convertirá en una Linked List.
4
5 AVL vs Red-Black Tree:
6   - AVL es más balanceado (búsqueda más rápida).
7   - Red-Black es más rápido para insertar/eliminar (menos
   rotaciones).
8   - `std::map` en C++ suele usar Red-Black.
9
10 Implementación: Casi nunca te pedirán programar un AVL completo
11 en una entrevista de 45 min (es muy largo), pero DEBES saber
   explicar
12 las rotaciones y por qué son necesarias.

```

Resumen rápido

- BST que se auto-balancea solo.
- Balance Factor: diferencia de altura entre hijos (máximo 1).
- Rotaciones: El truco místico para equilibrar el árbol.
- Garantiza $O(\log n)$ para todas las operaciones.
- Más complejo de programar que un BST normal.
- Ideal para aplicaciones donde hay MUCHAS búsquedas y pocas inserciones.

Capítulo 10

🌳 Recorridos de Árboles (Tree Traversals)

🤖 ¿Qué es?

Visitar todas las habitaciones de una casa de varios pisos . Hay dos grandes estilos:

- En profundidad (DFS): entras por una puerta y sigues bajando hasta el sótano antes de regresar. Según CUÁNDO anotes cada habitación (al entrar, en medio, o al salir) obtienes Preorder, Inorder o Postorder.
- Por niveles (BFS): recorres piso por piso: primero toda la planta baja, luego todo el primer piso, y así. Eso es Level-order.

El "cuándo apuntas el nodo" cambia por completo el orden del resultado.

💬 Dicho de otra forma

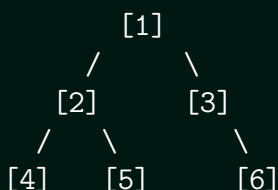
Recorrer un árbol es visitar cada nodo exactamente una vez. Los cuatro recorridos fundamentales:

- **Preorder** (Raíz $\rightarrow$ Izq $\rightarrow$ Der): útil para COPIAR/serializar un árbol.
- **Inorder** (Izq $\rightarrow$ Raíz $\rightarrow$ Der): en un BST devuelve los valores ORDENADOS.
- **Postorder** (Izq $\rightarrow$ Der $\rightarrow$ Raíz): útil para BORRAR el árbol o evaluar expresiones (procesa hijos antes que el padre).
- **Level-order** (por niveles): BFS con Queue; base de muchos problemas de árboles en entrevistas.

Los tres DFS salen naturales con recursión, pero conviene dominar la versión iterativa con Stack (evita stack overflow y la piden mucho).

🧠 Diagrama

Árbol:



```

Preorder  (R,I,D): 1  2  4  5  3  6
Inorder   (I,R,D): 4  2  5  1  3  6
Postorder (I,D,R): 4  5  2  6  3  1
Level-order : 1  2  3  4  5  6    (por niveles, con Queue)

```

```

1 DFS RECURSIVO
2 - Preorder : procesa nodo, luego izq, luego der.
3 - Inorder  : izq, procesa nodo, der.
4 - Postorder: izq, der, procesa nodo.
5
6 LEVEL-ORDER (BFS)
7 - Queue con la raíz. Saca un nodo, procésalo, encola sus hijos.
8 - Para separar por niveles: procesa exactamente queue.size() nodos
9   por iteración del bucle externo.
10
11 PSEUDOCÓDIGO (level-order por niveles)
12
13
14 función levelOrder(root):
15     SI root == NULL: return
16     q = Queue con root
17     MIENTRAS q no vacia:
18         n = q.size()                // nodos de este nivel
19         REPETIR n veces:
20             nodo = q.dequeue()
21             procesar(nodo)
22             SI nodo.izq: q.enqueue(nodo.izq)
23             SI nodo.der: q.enqueue(nodo.der)
24         // aqui termina un nivel
25
26 CÓDIGO C++ (DFS recursivo)
27
28
29 struct Node { int val; Node *left, *right; };
30
31 void preorder(Node* n) { if(!n) return; cout<<n->val<<" ";
32                          preorder(n->left); preorder(n->right); }
33 void inorder(Node* n)  { if(!n) return; inorder(n->left);
34                          cout<<n->val<<" "; inorder(n->right); }
35 void postorder(Node* n){ if(!n) return; postorder(n->left);
36                          postorder(n->right); cout<<n->val<<" "; }
37
38 CÓDIGO C++ (inorder iterativo con Stack)
39
40
41 #include <stack>
42 #include <vector>

```

```

43 using namespace std;
44
45 vector<int> inorderIter(Node* root) {
46     vector<int> res; stack<Node*> st; Node* cur = root;
47     while (cur || !st.empty()) {
48         while (cur) { st.push(cur); cur = cur->left; } // baja a la
49         // izq
49         cur = st.top(); st.pop();
50         res.push_back(cur->val); // procesa
51         cur = cur->right; // ve a la
52         // der
52     }
53     return res;
54 }
55
56 CÓDIGO C++ (level-order con Queue)
57
58
59 #include <queue>
60 vector<vector<int>> levelOrder(Node* root) {
61     vector<vector<int>> res;
62     if (!root) return res;
63     queue<Node*> q; q.push(root);
64     while (!q.empty()) {
65         int n = q.size();
66         vector<int> nivel;
67         for (int i = 0; i < n; i++) {
68             Node* cur = q.front(); q.pop();
69             nivel.push_back(cur->val);
70             if (cur->left) q.push(cur->left);
71             if (cur->right) q.push(cur->right);
72         }
73         res.push_back(nivel);
74     }
75     return res;
76 }
77
78 COMPLEJIDAD (Big-O)
79
80
81 Recorrido      Tiempo      Espacio
82
83 Los 4          O(n)        O(h) DFS / O(w) BFS
84
85 * n = nodos, h = altura, w = anchura máxima del árbol.

```

En entrevistas

Inorder de un BST = ordenado: base para validar un BST o encontrar el k-ésimo menor. Si el inorder no sale creciente, NO es un BST válido.

Level-order es la puerta a decenas de problemas: "Right Side View", "Zigzag Traversal", "conectar nodos del mismo nivel", "profundidad mínima". Casi siempre es BFS por niveles con `queue.size()`.

Recursión vs iterativo: la recursiva es más limpia, pero con árboles muy profundos (miles de niveles) puede desbordar el stack. Ten la versión iterativa lista.

Reconstrucción: con Preorder + Inorder (o Postorder + Inorder) puedes reconstruir un árbol binario único. Pregunta frecuente de Google.

Resumen rápido

- Preorder (R,I,D): copiar/serializar.
- Inorder (I,R,D): en BST da orden creciente.
- Postorder (I,D,R): borrar árbol, evaluar expresiones.
- Level-order (BFS con Queue): recorridos por niveles.
- Domina la versión iterativa con Stack (evita stack overflow).
- Todos $O(n)$ tiempo; espacio $O(h)$ en DFS, $O(\text{ancho})$ en BFS.

Capítulo 11

Ancestro Común (LCA) y Serialización de Árboles

¿Qué es?

Dos primos buscando a su pariente compartido más cercano en un árbol genealógico. El Ancestro Común más Bajo (Lowest Common Ancestor) es la primera persona de la que ambos descienden: ni muy arriba (el bisabuelo de todos), ni imposible (uno mismo).

Y la Serialización es tomarle una "foto plana" al árbol para guardarlo en un archivo o mandarlo por la red, de forma que después puedas RECONSTRUIRLO idéntico. Como desarmar un mueble con instrucciones para volverlo a armar exactamente igual.

Dicho de otra forma

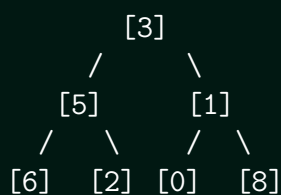
El **LCA** de dos nodos p y q es el nodo más profundo que tiene a ambos como descendientes (un nodo puede ser descendiente de sí mismo).

- En un **BST**: aprovecha el orden. Baja desde la raíz; si ambos valores son menores ve a la izquierda, si ambos son mayores ve a la derecha; en cuanto se "separan", ese nodo es el LCA.
- En un **árbol binario general**: DFS recursivo. Si encuentras p o q los devuelves; el primer nodo que recibe respuesta distinta de NULL por AMBOS lados es el LCA.

La **serialización** convierte el árbol en una cadena (típicamente un preorder con marcadores para los NULL) y la deserialización la reconstruye. Es la base de guardar árboles en disco o red.

Diagrama

Árbol:



```

LCA(6, 2) = 5      (ambos cuelgan de 5)
LCA(6, 8) = 3      (se separan en la raíz)
LCA(5, 2) = 5      (un nodo es ancestro de sí mismo)

```

Serialización (preorder, # = NULL):

```
3,5,6,#,#,2,#,#,1,0,#,#,8,#,#
```

Deserializar lee en el mismo orden y reconstruye idéntico.

```

1 LCA EN ÁRBOL BINARIO GENERAL (DFS)
2 1. Caso base: si el nodo es NULL o es p o q, devuélvelo.
3 2. Busca en izquierda y en derecha.
4 3. Si ambos lados devuelven algo -> este nodo es el LCA.
5 4. Si solo un lado devuelve algo -> propaga ese resultado hacia
   arriba.
6
7 PSEUDOCÓDIGO (LCA general)
8
9
10 función lca(nodo, p, q):
11     SI nodo == NULL O nodo == p O nodo == q: return nodo
12     izq = lca(nodo.izq, p, q)
13     der = lca(nodo.der, p, q)
14     SI izq != NULL Y der != NULL: return nodo    // p y q en lados
        distintos
15     return (izq != NULL) ? izq : der
16
17 CÓDIGO C++ (LCA general y en BST)
18
19
20 struct Node { int val; Node *left, *right; };
21
22 // Árbol binario general -> O(n)
23 Node* lca(Node* root, Node* p, Node* q) {
24     if (!root || root == p || root == q) return root;
25     Node* l = lca(root->left, p, q);
26     Node* r = lca(root->right, p, q);
27     if (l && r) return root;           // se separan aquí -> LCA
28     return l ? l : r;
29 }
30
31 // BST -> O(h), aprovecha el orden
32 Node* lcaBST(Node* root, int p, int q) {
33     while (root) {
34         if (p < root->val && q < root->val)    root = root->left;
35         else if (p > root->val && q > root->val) root = root->right;
36         ;
37         else return root;                 // se separan -> LCA
38     }
39     return nullptr;

```

CÓDIGO C++ (serializar / deserializar, preorder)

```
#include <string>
#include <sstream>
using namespace std;

void serializeH(Node* n, string& out) {
    if (!n) { out += "#,"; return; }
    out += to_string(n->val) + ",";
    serializeH(n->left, out);
    serializeH(n->right, out);
}

Node* deserializeH(istringstream& in) {
    string tok; getline(in, tok, ',');
    if (tok == "#") return nullptr;
    Node* n = new Node{stoi(tok), nullptr, nullptr};
    n->left = deserializeH(in);
    n->right = deserializeH(in);
    return n;
}
```

COMPLEJIDAD (Big-O)

Operación	Tiempo	Espacio
LCA (general)	$O(n)$	$O(h)$
LCA (BST)	$O(h)$	$O(1)$
Serializar	$O(n)$	$O(n)$
Deserializar	$O(n)$	$O(n)$

En entrevistas

”Lowest Common Ancestor” (LeetCode 235 BST / 236 general): dos versiones. En BST usa el orden (más rápido, $O(h)$); en árbol general usa el DFS de ”ambos lados devuelven algo”.

”Serialize and Deserialize Binary Tree” (LeetCode 297): clave usar un marcador para NULL (#) para que la reconstrucción sea única.

Sin marcador de NULL no se puede reconstruir: un preorder de solo valores es ambiguo. Los # son obligatorios.

Distancia entre dos nodos = $\text{profundidad}(p) + \text{profundidad}(q) - 2 \cdot \text{profundidad}(\text{LCA})$. El LCA es la pieza para medir distancias en árboles.

Resumen rápido

- LCA: ancestro común más profundo de dos nodos.
- BST: usa el orden, baja hasta que p y q se separan; $O(h)$.
- General: DFS; el nodo donde ambos lados devuelven algo es el LCA.
- Serializar: preorder con # para los NULL; $O(n)$.
- Deserializar lee en el mismo orden y reconstruye idéntico.
- LCA es la base para medir distancias entre nodos.

Capítulo 12



Heaps (Montículos)

🤖 ¿Qué es?

Un torneo de fútbol donde el mejor equipo SIEMPRE llega a la final (está en la cima).

Si el mejor equipo pierde o se retira (lo sacas), el siguiente mejor sube automáticamente a ocupar su lugar. No sabes exactamente quién es el tercer o cuarto mejor sin hacer más comparaciones, pero SIEMPRE tienes al #1 disponible de inmediato.

Es un árbol binario especial donde la raíz siempre tiene el valor máximo (Max-Heap) o mínimo (Min-Heap) de toda la estructura.

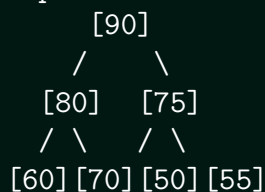
💬 Dicho de otra forma

Un Heap es un árbol binario completo (lleno de arriba a abajo, de izquierda a derecha) que cumple la "Propiedad de Heap": - Max-Heap: El padre siempre es mayor o igual que sus hijos. - Min-Heap: El padre siempre es menor o igual que sus hijos.

A diferencia del BST, no hay orden entre hermanos. Lo único que importa es la relación Padre-Hijo. Por su estructura compacta, se suele implementar eficientemente usando un Array.

🤖 Diagrama

Max-Heap Visual:



Implementado como Array (¡Truco de índices!):

Idx: [0] [1] [2] [3] [4] [5] [6]

Val: [90] [80] [75] [60] [70] [50] [55]

Fórmulas mágicas para el índice `i`:

- Parent: $(i - 1) / 2$

- Left Child: $2 * i + 1$

```
- Right Child: 2 * i + 2
```

1. Insert: Agregas el nuevo valor al final del array (la hoja más a la izquierda disponible). Luego lo comparas con su padre y lo subes si es necesario (Bubble Up / Heapify Up). $O(\log n)$.
2. Extract Max/Min: Sacas la raíz ($\text{arr}[0]$). Mueves el último elemento a la raíz y lo vas bajando comparándolo con sus hijos hasta que esté en su lugar (Bubble Down / Heapify Down). $O(\log n)$.
3. Peek: Miras $\text{arr}[0]$. $O(1)$.
4. Heapify (Construir Heap): Convertir un array desordenado en un Heap. $O(n)$.

PSEUDOCÓDIGO

```
// Heapify Up (al insertar)
función subir(i):
    MIENTRAS i > 0 Y arr[padre(i)] < arr[i]:
        intercambiar(arr[padre(i)], arr[i])
        i = padre(i)

// Heapify Down (al extraer)
función bajar(i):
    mayor = i
    SI hijoIzquierdo(i) existe Y arr[hijoIzquierdo(i)] > arr[mayor]:
        mayor = hijoIzquierdo(i)
    SI hijoDerecho(i) existe Y arr[hijoDerecho(i)] > arr[mayor]:
        mayor = hijoDerecho(i)
    SI mayor != i:
        intercambiar(arr[i], arr[mayor])
        bajar(mayor)
```

CÓDIGO C++

```
#include <iostream>
#include <vector>
#include <queue> // STL priority_queue es un Heap
using namespace std;

int main() {
    // 1. Max-Heap (por defecto)
    priority_queue<int> maxHeap;

    maxHeap.push(10);
    maxHeap.push(30);
    maxHeap.push(20);

    cout << "Máximo en la cima: " << maxHeap.top() << endl; // 30
```

```

48
49 maxHeap.pop(); // Quita el 30
50 cout << "Nuevo máximo: " << maxHeap.top() << endl; // 20
51
52 // 2. Min-Heap
53 priority_queue<int, vector<int>, greater<int>> minHeap;
54 minHeap.push(10);
55 minHeap.push(30);
56 minHeap.push(20);
57
58 cout << "Mínimo en la cima: " << minHeap.top() << endl; // 10
59
60 return 0;
61 }

```

COMPLEJIDAD (Big-O)

Operación	Tiempo	Razón
Insert	$O(\log n)$	Altura del árbol
Extract Max/Min	$O(\log n)$	Altura del árbol
Peek Max/Min	$O(1)$	Es la raíz (arr[0])
Heapify (Build)	$O(n)$	Análisis matemático avanzado
Search	$O(n)$	No es un BST, hay que buscar todo

"Top K Elements": Si te piden los K elementos más grandes, usa un Min-Heap de tamaño K. Si te piden los más chicos, usa un Max-Heap.

Priority Queue = Heap: En el 99% de los casos, cuando necesites un Heap en una entrevista, usa `std::priority_queue`. No lo programes desde cero a menos que te lo pidan explícitamente.

No es un BST: Recuerda que un Heap NO está ordenado de izquierda a derecha. Solo garantiza la relación Padre > Hijo.

Heapsort: Puedes ordenar un array metiendo todo a un Heap y sacándolo uno por uno. $O(n \log n)$.

Resumen rápido

- Árbol binario completo donde el jefe (raíz) es el mayor o menor.
- Implementado como Array para máxima velocidad y ahorro de memoria.
- Insertar y sacar el máximo/mínimo es $O(\log n)$.
- Ver el máximo/mínimo es instantáneo $O(1)$.
- Usos: Algoritmo de Dijkstra, colas de prioridad, problemas de "Top K".
- Heapify (construir desde array) es sorprendentemente $O(n)$.

Capítulo 13

Hash Tables (Tablas Hash / Mapas)

¿Qué es?

Un casillero de escuela . Cada estudiante tiene un número de casillero asignado por una fórmula mágica (la Hash Function).

Para guardar la mochila de "Carlos", aplicas la fórmula: $\text{Hash}(\text{"Carlos"}) = 42$. Vas directo al casillero 42 y la guardas. Para buscarla, vuelves a aplicar la fórmula: $\text{Hash}(\text{"Carlos"}) = 42$, y vas DIRECTO al casillero 42.

¡No tienes que revisar todos los casilleros uno por uno! Es como tener un acceso instantáneo a cualquier dato si conoces su "llave".

Dicho de otra forma

Una Hash Table es una estructura que asocia Llaves (Keys) con Valores (Values). Usa una función matemática para convertir la Key en un índice de un array.

El objetivo principal es lograr operaciones de búsqueda, inserción y eliminación en tiempo constante promedio $O(1)$. Es la estructura más eficiente para búsquedas rápidas en la industria.

Diagrama

```
Key: "Carlos"  --> [ Hash Function ] --> Index: 2
Key: "Maria"   --> [ Hash Function ] --> Index: 5
```

Array Interno (Buckets):

```
[0] : NULL
[1] : NULL
[2] : {Key: "Carlos", Value: "Mochila Azul"}
[3] : NULL
[4] : NULL
[5] : {Key: "Maria", Value: "Mochila Roja"}
```

Colisión (Collision):

Si $\text{Hash}(\text{"Juan"})$ también da 2, ocurre una colisión.

Solución común: Chaining (una Linked List en el índice 2).

```
[2] : ["Carlos"] -> ["Juan"] -> NULL
```

1. Hash Function: Convierte cualquier Key (string, objeto, etc.) en un número entero dentro del rango del array.
2. Insert (set): Calculas el hash, vas al índice y guardas el par {K,V}.
3. Search (get): Calculas el hash, vas al índice y devuelves el valor.
4. Delete: Calculas el hash y eliminas la entrada en ese índice.
5. Rehashing: Si la tabla se llena mucho (Load Factor > 0.75), se crea un array más grande y se vuelven a calcular todos los hashes.

PSEUDOCÓDIGO

```
// Insertar en un Hash Map (con Chaining)
función insertar(llave, valor):
    índice = hash(llave) % tamaño_tabla
    lista = buckets[índice]
    SI llave ya existe en lista:
        actualizar valor
    SINO:
        agregar {llave, valor} al final de la lista
```

CÓDIGO C++

```
#include <iostream>
#include <unordered_map> // Hash Map estándar en C++
#include <unordered_set> // Hash Set (solo llaves únicas)
#include <string>
using namespace std;

int main() {
    // 1. Declaración
    unordered_map<string, int> edades;

    // 2. Insertar -> O(1) promedio
    edades["Carlos"] = 25;
    edades["Maria"] = 30;
    edades.insert({"Juan", 22});

    // 3. Acceder -> O(1) promedio
    cout << "Edad de Maria: " << edades["Maria"] << endl;

    // 4. Verificar existencia
    if (edades.find("Pedro") == edades.end()) {
        cout << "Pedro no está en la tabla." << endl;
    }
}
```

```

47 // 5. Eliminar -> O(1)
48 edades.erase("Carlos");
49
50 // 6. Recorrer
51 for (auto const& [nombre, edad] : edades) {
52     cout << nombre << " tiene " << edad << " años." << endl;
53 }
54
55 return 0;
56 }
57
58 COMPLEJIDAD (Big-O)
59
60
61 Operación      Promedio      Peor caso
62
63 Insert         O(1)           O(n)
64 Search         O(1)           O(n)
65 Delete         O(1)           O(n)
66 Espacio        O(n)           O(n)
67
68 * El peor caso O(n) ocurre si la función hash es muy mala y todos
69   los
   elementos caen en el mismo casillero (colisión masiva).

```

En entrevistas

Regla de Oro: Si necesitas buscar algo en $O(1)$, LA respuesta es un Hash Map o Hash Set.

"Two Sum": El problema más famoso que se resuelve con Hash Map. Guardas el complemento de lo que buscas para encontrarlo en $O(1)$.

map vs unordered_map (C++): - 'unordered_map': Usa Hash Table, $O(1)$ promedio, no tiene orden. - 'map': Usa Red-Black Tree, $O(\log n)$, mantiene las llaves ordenadas.

Collision Handling: Debes saber explicar Chaining (listas) y Open Addressing (buscar el siguiente hueco libre).

Resumen rápido

- Key -> Hash Function -> Index.
- Acceso, búsqueda e inserción instantáneos $O(1)$.
- La función hash debe ser rápida y distribuir bien los datos.
- Colisiones: Cuando dos llaves quieren el mismo lugar.
- Load Factor: Qué tan llena está la tabla.
- Estructura más usada en la vida real por su velocidad.

Capítulo 14

✧ Hashing a Fondo: Colisiones e Implementación

🤖 ¿Qué es?

Un estacionamiento donde una fórmula te asigna el cajón según tu placa. El problema: a veces dos coches distintos reciben el MISMO cajón. Eso es una colisión, y toda tabla hash necesita un plan para resolverla:

- Chaining: en cada cajón cabe una fila de coches (una lista). Si chocan, se forman detrás.
- Open Addressing: si tu cajón está ocupado, buscas el siguiente libre siguiendo una regla.

Y si el estacionamiento se llena demasiado (Load Factor alto), se construye uno más grande y se reacomodan todos los coches (rehashing).

💬 Dicho de otra forma

Una hash table promedia $O(1)$ SOLO si las colisiones se manejan bien. Dos grandes estrategias:

Chaining (encadenamiento): cada bucket guarda una lista enlazada de pares que colisionaron. Simple y tolerante a Load Factors altos. Es lo que usa `std::unordered_map`.

Open Addressing (direccionamiento abierto): todo vive en el array; ante colisión se busca otro hueco mediante probing:

- Linear probing: prueba $i + 1, i + 2, \dots$ (sufrir "clustering").
- Quadratic probing: prueba $i + 1, i + 4, i + 9, \dots$
- Double hashing: el salto lo decide una segunda función hash.

El **Load Factor** $\alpha = n/m$ (elementos / buckets) mide qué tan llena está la tabla. Cuando supera un umbral (0.75), se hace **rehashing**: se duplica el array y se reinsertan todas las claves.

🧠 Diagrama

CHAINING (listas por bucket)

[0] -> NULL

OPEN ADDRESSING (linear probing)

hash("A")=2, hash("B")=2, hash("C")=3

```

[1] -> NULL
[2] -> (A) -> (B) -> (C)      [0] -
[3] -> NULL                    [1] -
[4] -> (D)                     [2] A    <- va a su hueco
                                [3] B    <- ocupado el 2, prueba 3? no,
                                C    <- desplazados en cadena
Colisiones -> se apilan en la  [4] -
lista del bucket.

Load Factor a = n/m ; si a > 0.75 -> rehash (duplicar m y reinsertar).

```

```

1 IMPLEMENTACIÓN PROPIA (chaining)
2 1. Array de buckets; cada bucket es una lista de pares {clave,
3   valor}.
4 2. índice = hash(clave) % capacidad.
5 3. Insert: si la clave existe en la lista, actualiza; si no, agré
6   gala.
7 4. Get: recorre la lista del bucket buscando la clave.
8 5. Si load factor > umbral -> rehash (nueva capacidad, reinsertar
9   todo).
10
11 PSEUDOCÓDIGO (get con chaining)
12
13 función get(clave):
14     idx = hash(clave) % capacidad
15     PARA cada par en buckets[idx]:
16         SI par.clave == clave: return par.valor
17     return NO_ENCONTRADO
18
19 CÓDIGO C++ (hash map propio con chaining)
20
21 #include <vector>
22 #include <list>
23 #include <utility>
24 using namespace std;
25
26 class MyHashMap {
27     vector<list<pair<int,int>>> buckets;
28     int capacity, size_;
29     int idx(int key) { return (key % capacity + capacity) %
30         capacity; }
31
32     void rehash() {
33         auto old = buckets;
34         capacity *= 2;
35         buckets.assign(capacity, {});
36         size_ = 0;
37         for (auto& b : old) for (auto& kv : b) put(kv.first, kv.

```

```

        second);
    }
public:
    MyHashMap(int cap = 8) : buckets(cap), capacity(cap), size_(0)
    {}

    void put(int key, int val) {
        auto& b = buckets[idx(key)];
        for (auto& kv : b) if (kv.first == key) { kv.second = val;
            return; }
        b.push_back({key, val});
        if (++size_ > capacity * 0.75) rehash();    // load factor
    }
    int get(int key) {
        for (auto& kv : buckets[idx(key)])
            if (kv.first == key) return kv.second;
        return -1;    // no encontrado
    }
    void remove(int key) {
        auto& b = buckets[idx(key)];
        for (auto it = b.begin(); it != b.end(); ++it)
            if (it->first == key) { b.erase(it); size_--; return; }
    }
};

COMPLEJIDAD (Big-O)

Método          Promedio   Peor caso

Insert/Get/Delete  O(1)       O(n)
Rehash            O(n)       O(n) (amortizado O(1) por op)

* Peor caso O(n): función hash mala -> todo cae en un bucket.

```

🔗 En entrevistas

Chaining vs Open Addressing: chaining tolera load factors altos y borra fácil; open addressing usa mejor la cache y menos memoria por puntero, pero se degrada al llenarse. Sabe explicar ambos.

Una buena hash function distribuye uniforme y es rápida. Para strings, el hash polinómico ($h = h*31 + c$) es el clásico.

Borrar en open addressing es delicado: no puedes dejar el hueco vacío (rompería las búsquedas por probing); se marca como "tombstone" (borrado lógico).

`unordered_map` se degrada a $O(n)$: con claves maliciosas que colisionan a propósito (hash flooding). Por eso algunos lenguajes aleatorizan la semilla del hash.

Resumen rápido

- Colisión: dos claves caen en el mismo bucket. Toda tabla necesita plan.
- Chaining: lista por bucket (lo usa unordered_map).
- Open Addressing: buscar otro hueco (linear/quadratic/double hashing).
- Load Factor $\alpha = n/m$; si pasa 0.75 -> rehash (duplicar y reinsertar).
- Promedio $O(1)$; peor caso $O(n)$ con hash mala.
- Borrar en open addressing usa tombstones.

Capítulo 15

🔑 Patrones con Hash Map

🤖 ¿Qué es?

El Hash Map es la navaja suiza de las entrevistas: cuando algo tarda $O(n^2)$ "buscando de nuevo", casi siempre un Hash Map lo baja a $O(n)$ recordando lo que ya viste.

Tres patrones que aparecen una y otra vez:

- Contar frecuencias: cuántas veces aparece cada cosa.
- Complemento (Two Sum): guardar lo que YA viste para encontrar su pareja al instante.
- Prefix-Sum + Hash: contar sub-arreglos cuya suma es un objetivo, recordando las sumas acumuladas.

💬 Dicho de otra forma

El Hash Map convierte "¿ya vi esto antes?" en una consulta $O(1)$. Los patrones más rentables:

- **Frequency Count**: un mapa valor $\rightarrow$ conteo. Base de anagramas, mayoría, top-K, duplicados.
- **Two Sum / complemento**: por cada x busca si $target - x$ ya está en el mapa. Una sola pasada, $O(n)$.
- **Group By (agrupación)**: mapa clave_canónica $\rightarrow$ lista. Para anagramas la clave es la palabra ordenada o su vector de conteo de letras.
- **Prefix-Sum + Hash**: para "subarreglos que suman K", guarda cuántas veces apareció cada suma acumulada; si suma - K ya se vio, hay subarreglos que suman K.

🧠 Diagrama

TWO SUM (target = 9)

nums = [2, 7, 11, 15]

visto = {}

x=2 $\rightarrow$ falta 7? no. guarda 2

x=7 $\rightarrow$ falta 2? SÍ (índice 0)
 $\Rightarrow$ par (0,1)

SUBARRAY SUM = K (K = 3)

nums = [1, 2, 1, 3], K = 3

prefix acumulada: 1, 3, 4, 7

mapa {suma_prefija $\rightarrow$ veces}, inicia {0:1}

suma=1 $\rightarrow$ busca 1-3=-2? no. mapa{0:1,1:1}

suma=3 $\rightarrow$ busca 3-3=0? SÍ (1 vez) $\rightarrow$ +1

suma=4 $\rightarrow$ busca 4-3=1? SÍ (1 vez) $\rightarrow$ +1

```

suma=7 -> busca 7-3=4? SÍ (1 vez) -> +1
total subarreglos con suma 3: 3

```

```

1 TWO SUM
2 - Un mapa valor->índice. Por cada x, si target-x ya está,
   encontraste
3   el par. Si no, guarda x. Una sola pasada.
4
5 SUBARRAY SUM = K
6 - Suma acumulada 'suma'. Un mapa 'cuantas veces salió cada suma'.
7 - Si (suma - K) está en el mapa, esos son subarreglos que suman K.
8 - Inicializa el mapa con {0: 1} (el prefijo vacío).
9
10 PSEUDOCÓDIGO (subarray sum = K)
11
12
13 función subarraysConSuma(nums, K):
14     conteo = {0: 1}; suma = 0; total = 0
15     PARA cada x en nums:
16         suma += x
17         SI (suma - K) en conteo: total += conteo[suma - K]
18         conteo[suma] += 1
19     return total
20
21 CÓDIGO C++ (Two Sum)
22
23
24 #include <vector>
25 #include <unordered_map>
26 using namespace std;
27
28 vector<int> twoSum(vector<int>& nums, int target) {
29     unordered_map<int,int> visto;           // valor -> índice
30     for (int i = 0; i < (int)nums.size(); i++) {
31         int falta = target - nums[i];
32         if (visto.count(falta)) return {visto[falta], i};
33         visto[nums[i]] = i;
34     }
35     return {};
36 }
37
38 CÓDIGO C++ (subarray sum = K)
39
40
41 int subarraySum(vector<int>& nums, int k) {
42     unordered_map<int,int> conteo{{0, 1}}; // prefijo vacío
43     int suma = 0, total = 0;
44     for (int x : nums) {
45         suma += x;
46         if (conteo.count(suma - k)) total += conteo[suma - k];
47         conteo[suma]++;

```

```

48     }
49     return total;
50 }
51
52 CÓDIGO C++ (agrupar anagramas)
53
54
55 #include <string>
56 #include <algorithm>
57 vector<vector<string>> groupAnagrams(vector<string>& strs) {
58     unordered_map<string, vector<string>> grupos;
59     for (auto& s : strs) {
60         string clave = s;
61         sort(clave.begin(), clave.end());           // clave canónica
62         grupos[clave].push_back(s);
63     }
64     vector<vector<string>> res;
65     for (auto& [k, v] : grupos) res.push_back(v);
66     return res;
67 }
68
69 COMPLEJIDAD (Big-O)
70
71
72 Patrón                Tiempo                Espacio
73
74 Frequency / TwoSum    O(n)                    O(n)
75 Subarray sum = K      O(n)                    O(n)
76 Group anagrams        O(n·k log k)            O(n·k)
77
78 * k = longitud de la palabra en group anagrams.

```

En entrevistas

Regla mental: si te descubres haciendo un bucle DENTRO de otro para "volver a buscar", pregúntate "¿un Hash Map recuerda esto en $O(1)$?". Suele bajar $O(n^2)$ a $O(n)$.

Prefix-Sum + Hash es el patrón estrella oculto: "Subarray Sum Equals K" (LeetCode 560), "Continuous Subarray Sum", "Contiguous Array". Inicializar el mapa con $\{0: 1\}$ es el detalle que a todos se les olvida.

Group anagrams por conteo de letras: en vez de ordenar ($k \log k$), usa un vector de 26 conteos como clave $\rightarrow O(k)$. Mejora que impresiona.

Hash Set vs Hash Map: si solo importa "¿existe?" usa un Set; si necesitas asociar un dato (índice, conteo) usa un Map.

Resumen rápido

- Hash Map convierte "¿ya lo vi?" en $O(1)$: rompe bucles $O(n^2)$.
- Frequency count: base de anagramas, mayoría, duplicados, top-K.
- Two Sum: guarda visto, busca el complemento $\text{target} - x$.
- Prefix-Sum + Hash: subarreglos con suma K; inicia el mapa con 0:1.
- Group by: mapa de clave canónica $\rightarrow$ lista.
- Set si solo importa existencia; Map si asocias datos.

Capítulo 16

🔗 LRU Cache (Hash Map + Lista Doblemente Enlazada)

🤖 ¿Qué es?

Tu escritorio solo tiene espacio para 5 carpetas. Cada vez que usas una, la pones ENCIMA (la más reciente). Cuando llega una carpeta nueva y ya no cabe, tiras la de HASTA ABAJO: la que llevas más tiempo sin tocar ("Least Recently Used", la menos usada recientemente).

Una LRU Cache hace justo eso con datos, y el reto es lograr que TODO (guardar, buscar, mover a la cima y tirar la vieja) sea instantáneo $O(1)$. La combinación mágica: un Hash Map para encontrar rápido + una lista doblemente enlazada para reordenar rápido.

💬 Dicho de otra forma

Una LRU (Least Recently Used) Cache guarda hasta **capacity** pares clave-valor y, al llenarse, desaloja el elemento usado hace más tiempo. Debe hacer **get** y **put** en $O(1)$.

Ninguna estructura sola lo logra, así que se combinan dos:

- **Hash Map** **clave** -> **nodo**: encuentra cualquier elemento en $O(1)$.
- **Lista doblemente enlazada**: mantiene el orden de uso. El frente es el más reciente; el fondo, el candidato a desalojar. Mover un nodo o quitarlo es $O(1)$ porque tienes punteros a ambos vecinos.

Cada acceso (**get** o **put**) mueve el nodo al frente. Al exceder la capacidad, se elimina el nodo del fondo (y su clave del mapa).

🧠 Diagrama

Capacidad = 3. Frente = más reciente, Fondo = menos reciente.

```
put(1) put(2) put(3):  
FRENTE [3] <-> [2] <-> [1] FONDO  
mapa: {1:n1, 2:n2, 3:n3}
```

```
get(1) -> mueve 1 al frente:  
FRENTE [1] <-> [3] <-> [2] FONDO
```

```

put(4) -> lleno! desaloja el fondo (2):
FRENTE [4] <-> [1] <-> [3] FONDO
mapa: {1,3,4} (el 2 se eliminó del mapa y de la lista)

```

Nodos "centinela" (head/tail falsos) evitan casos borde con NULL.

```

1  CÓMO FUNCIONA (todo O(1))
2  - Hash Map clave -> puntero al nodo de la lista.
3  - Lista doble con centinelas head/tail falsos.
4  - get(k): si existe, mueve su nodo al frente y devuelve el valor.
5  - put(k,v): si existe, actualiza y mueve al frente; si no, crea al
6    frente. Si superas la capacidad, borra el nodo antes del tail.
7
8  PSEUDOCÓDIGO
9
10
11 get(clave):
12     SI clave no en mapa: return -1
13     nodo = mapa[clave]; moverAlFrente(nodo)
14     return nodo.valor
15
16 put(clave, valor):
17     SI clave en mapa:
18         nodo = mapa[clave]; nodo.valor = valor; moverAlFrente(nodo)
19     SINO:
20         SI tamaño == capacidad:
21             viejo = tail.prev; quitar(viejo); borrar mapa[viejo.
22                 clave]
23             nodo = nuevo Nodo(clave, valor)
24             insertarAlFrente(nodo); mapa[clave] = nodo
25
26 CÓDIGO C++
27
28 #include <unordered_map>
29 using namespace std;
30
31 class LRUCache {
32     struct Node { int key, val; Node *prev, *next; };
33     int cap;
34     unordered_map<int, Node*> mp;
35     Node *head, *tail; // centinelas
36
37     void remove(Node* n) { n->prev->next = n->next; n->next->prev =
38         n->prev; }
39     void addFront(Node* n) {
40         n->next = head->next; n->prev = head;
41         head->next->prev = n; head->next = n;
42     }
43 public:

```

```

43  LRUCache(int capacity) : cap(capacity) {
44      head = new Node(); tail = new Node();
45      head->next = tail; tail->prev = head;
46  }
47  int get(int key) {
48      if (!mp.count(key)) return -1;
49      Node* n = mp[key];
50      remove(n); addFront(n);          // marcar como reciente
51      return n->val;
52  }
53  void put(int key, int value) {
54      if (mp.count(key)) {
55          Node* n = mp[key]; n->val = value;
56          remove(n); addFront(n);
57          return;
58      }
59      if ((int)mp.size() == cap) {      // desalojar el menos
                                         reciente
60          Node* lru = tail->prev;
61          remove(lru); mp.erase(lru->key); delete lru;
62      }
63      Node* n = new Node{key, value, nullptr, nullptr};
64      addFront(n); mp[key] = n;
65  }
66 };

```

COMPLEJIDAD (Big-O)

Operación	Tiempo	Espacio
get	$O(1)$	$O(\text{capacity})$
put	$O(1)$	$O(\text{capacity})$

* El $O(1)$ es EL punto del problema; una lista simple daría $O(n)$.



En entrevistas

”LRU Cache” (LeetCode 146): pregunta top de Google/Meta/Amazon. La respuesta esperada es EXACTAMENTE Hash Map + lista doble. Menos que eso no da $O(1)$.

Nodos centinela (dummy head/tail): eliminan todos los casos borde de NULL al insertar/borrar. No los omitas: simplifican mucho el código.

¿Por qué lista DOBLE y no simple? Para borrar un nodo en $O(1)$ necesitas su nodo anterior. La lista simple obligaría a recorrer $O(n)$.

Variante LFU (Least Frequently Used): desaloja por frecuencia, no por recencia. Más difícil; suele necesitar un mapa extra de frecuencias. Menciónala si te la piden.

 Resumen rápido

- LRU: al llenarse, desaloja el elemento usado hace más tiempo.
- Hash Map (buscar $O(1)$) + lista doble (reordenar $O(1)$).
- Frente = más reciente; fondo = candidato a desalojar.
- get/put mueven el nodo al frente; al exceder capacidad, borra el fondo.
- Centinelas head/tail eliminan casos borde con NULL.
- Lista DOBLE es obligatoria para borrar en $O(1)$.

Capítulo 17

Tries (Árboles de Prefijos)

¿Qué es?

Un diccionario físico donde navegas letra por letra. Para buscar la palabra "GATO": 1. Vas a la sección G. 2. Dentro de G, buscas la A. 3. Dentro de GA, buscas la T. 4. Dentro de GAT, buscas la O. ¡Encontrado!

Cada nivel del árbol representa una letra. Si compartes el mismo inicio (prefijo), compartes los mismos nodos. Por ejemplo, "GATO" y "GOTA" comparten la 'G', pero luego se bifurcan.

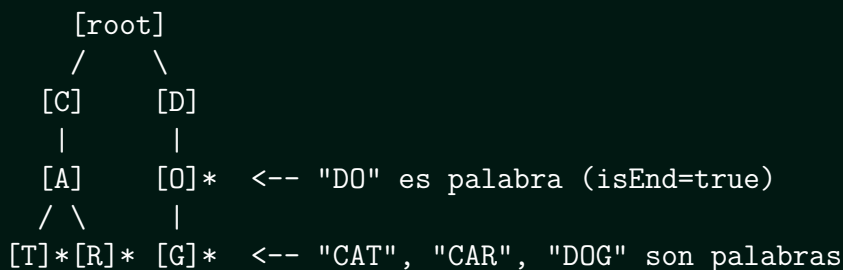
Dicho de otra forma

Un Trie (se pronuncia como "try") es un árbol especializado para guardar strings. Cada nodo representa un caracter.

Su gran ventaja es la eficiencia para búsquedas de prefijos y autocompletado. A diferencia de un Hash Map, el tiempo de búsqueda en un Trie depende únicamente de la longitud de la palabra (L), no del número de palabras guardadas (N).

Diagrama

Insertando: "CAT", "CAR", "DO", "DOG"



* : Indica el final de una palabra (isEnd = true).

1. insert(word): Empiezas en el root. Para cada letra, si no existe el hijo, lo creas. Al final de la palabra, marcas `isEnd = true`.
2. search(word): Sigues el camino de las letras. Si llegas al final y el nodo tiene `isEnd = true`, la palabra existe.
3. startsWith(prefix): Igual que search, pero no importa si `isEnd`

es **true**; con que el camino exista, el prefijo es válido. $O(L)$.

PSEUDOCÓDIGO

// Insertar palabra en el Trie

función insertar(palabra):

 actual = root

 PARA cada letra EN palabra:

 SI actual.hijos[letra] NO existe:

 actual.hijos[letra] = nuevo Nodo()

 actual = actual.hijos[letra]

 actual.esFinDePalabra = VERDADERO

CÓDIGO C++

#include <iostream>

#include <unordered_map>

#include <string>

using namespace std;

struct TrieNode {

 unordered_map<char, TrieNode*> children;

 bool isEndOfWord = **false**;

};

class Trie {

private:

 TrieNode* root;

public:

 Trie() { root = **new** TrieNode(); }

// Insertar -> $O(L)$ donde L es la longitud de la palabra

void insert(string word) {

 TrieNode* curr = root;

for (char c : word) {

if (curr->children.find(c) == curr->children.end()) {

 curr->children[c] = **new** TrieNode();

 }

 curr = curr->children[c];

 }

 curr->isEndOfWord = **true**;

}

// Buscar palabra completa -> $O(L)$

bool search(string word) {

 TrieNode* curr = root;

for (char c : word) {

if (curr->children.find(c) == curr->children.end())

```

        return false;
57     curr = curr->children[c];
58 }
59     return curr->isEndOfWord;
60 }
61
62 // Buscar prefijo -> O(L)
63 bool startsWith(string prefix) {
64     TrieNode* curr = root;
65     for (char c : prefix) {
66         if (curr->children.find(c) == curr->children.end())
67             return false;
68         curr = curr->children[c];
69     }
70     return true;
71 };
72
73 int main() {
74     Trie t;
75     t.insert("gato");
76     t.insert("goma");
77
78     cout << "¿Existe gato? " << t.search("gato") << endl;    // 1 (
79         true)
80     cout << "¿Existe prefijo go? " << t.startsWith("go") << endl;
81         // 1 (true)
82     cout << "¿Existe gatito? " << t.search("gatito") << endl; // 0
83         (false)
84
85     return 0;
86 }
87
88 COMPLEJIDAD (Big-O)
89
90 Operación          Tiempo          Espacio
91 Insert             O(L)             O(ALPHABET_SIZE * L * N)
92 Search             O(L)             O(1)
93 StartsWith         O(L)             O(1)
94
95 * L = Longitud de la palabra.
96 * N = Número de palabras.
97 * Nota: El espacio puede ser grande si las palabras no comparten
    prefijos.

```

En entrevistas

Cuándo usar: Si el problema menciona "autocompletar", "diccionario", o "buscar todas las palabras que empiecen con...", usa un Trie.

Trie + DFS: Una combinación clásica para problemas como "Word Search II" (encontrar palabras en una cuadrícula de letras).

Hash Map vs Trie: - Hash Map: Busca la palabra completa en $O(1)$ promedio.
- Trie: Puede buscar prefijos y es más rápido si hay muchas colisiones en el hash map.

Memoria: Si te preocupa el espacio, menciona que se puede usar un "Compressed Trie" (Radix Tree) donde los nodos con un solo hijo se fusionan.

Resumen rápido

- Árbol de letras, nivel por nivel.
- Ideal para búsquedas de prefijos y autocompletado.
- Tiempo de búsqueda dependiente solo del largo de la palabra.
- 'startsWith' es su operación estrella.
- Puede usar mucha memoria, pero ahorra espacio si hay muchos prefijos compartidos.

Capítulo 18

Union-Find (Disjoint Set Union - DSU)

¿Qué es?

Los clubes de la escuela . Al principio, cada estudiante es su propio club (un conjunto de 1 persona).

Cuando dos personas se hacen amigas, sus clubes se UNEN. Para saber si dos personas están en el mismo club, sigues la cadena de amistades hasta llegar al "líder" del grupo. Si ambos tienen el mismo líder, entonces ¡están en el mismo club!

Es una estructura que gestiona grupos de elementos que no se traslapan y te permite unirlos o preguntar si están conectados muy rápido.

Dicho de otra forma

Union-Find (o DSU) es una estructura de datos que mantiene un conjunto de elementos divididos en varios conjuntos disjuntos. Soporta dos operaciones principales:

1. Find: Determina a qué conjunto pertenece un elemento (devuelve el "representante" o "líder" del conjunto). 2. Union: Une dos conjuntos en uno solo.

Con optimizaciones como "Path Compression" y "Union by Rank", estas operaciones se vuelven prácticamente instantáneas $O(\log(n))$.

Diagrama

Estado Inicial: {0} {1} {2} {3} {4}
(Cada uno apunta a sí mismo como líder)

Union(0, 1): {0, 1} {2} {3} {4}
[0] ← [1] (0 es el líder de 1)

Union(1, 2): {0, 1, 2} {3} {4}
[0] ← [1] ← [2]

Path Compression (al buscar el líder de 2):
[0] ← [1], [0] ← [2] (Ahora 2 apunta directo al líder 0)

1. Find(i): Sigue los punteros `parent` desde `i` hasta llegar al nodo que es su propio padre. Ese es el representante.
 - Optimización (Path Compression): Mientras subes, haces que cada nodo apunte directo al representante final.
2. Union(i, j): Encuentras los representantes de `i` y `j`. Si son diferentes, haces que uno apunte al otro.
 - Optimización (Union by Rank): Siempre haces que el árbol más bajito cuelgue del árbol más alto para mantenerlo balanceado.

PSEUDOCÓDIGO

```
// Find con Path Compression
función buscar(i):
    SI parent[i] == i: return i
    parent[i] = buscar(parent[i]) // ¡Compresión aquí!
    return parent[i]

// Union por Rango
función unir(i, j):
    raiz_i = buscar(i)
    raiz_j = buscar(j)
    SI raiz_i != raiz_j:
        SI rango[raiz_i] < rango[raiz_j]:
            parent[raiz_i] = raiz_j
        SINO:
            parent[raiz_j] = raiz_i
            SI rango[raiz_i] == rango[raiz_j]:
                rango[raiz_i]++
```

CÓDIGO C++

```
#include <iostream>
#include <vector>
using namespace std;

class UnionFind {
    vector<int> parent;
    vector<int> rank;
public:
    UnionFind(int n) {
        parent.resize(n);
        rank.resize(n, 0);
        for(int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]); // Path compression
    }
}
```

```

52
53 void unite(int i, int j) {
54     int root_i = find(i);
55     int root_j = find(j);
56     if (root_i != root_j) {
57         if (rank[root_i] < rank[root_j]) parent[root_i] =
58             root_j;
59         else if (rank[root_i] > rank[root_j]) parent[root_j] =
60             root_i;
61         else {
62             parent[root_i] = root_j;
63             rank[root_j]++;
64         }
65     }
66
67     bool connected(int i, int j) {
68         return find(i) == find(j);
69     }
70
71 int main() {
72     UnionFind uf(5);
73     uf.unite(0, 1);
74     uf.unite(2, 3);
75
76     cout << "¿0 y 1 conectados? " << uf.connected(0, 1) << endl; //
77         1 (true)
78
79     cout << "¿0 y 2 conectados? " << uf.connected(0, 2) << endl; //
80         0 (false)
81
82     uf.unite(1, 2);
83     cout << "¿0 y 2 conectados ahora? " << uf.connected(0, 2) <<
84         endl; // 1 (true)
85
86     return 0;
87 }

```

COMPLEJIDAD (Big-O)

Operación	Tiempo	En la práctica
Find	$O((n))$	Casi $O(1)$
Union	$O((n))$	Casi $O(1)$
Connected	$O((n))$	Casi $O(1)$
Espacio	$O(n)$	Para guardar parents/ranks

* (n) es la función inversa de Ackermann, que crece tan lento que para cualquier valor real de n , es menor a 5.

En entrevistas

Cuándo usar: Problemas de conectividad, componentes conectados en grafos, o detectar ciclos en grafos no dirigidos.

Alternativa a DFS/BFS: Para saber si dos nodos están conectados, Union-Find es a veces más rápido y fácil de implementar que un recorrido completo.

Optimización: NUNCA olvides la "Path Compression". Sin ella, Union-Find puede degradarse a $O(n)$.

Problema Clásico: "Number of Islands" o "Redundant Connection".

Resumen rápido

- Gestiona grupos de elementos y sus uniones.
- Operaciones: 'find' (quién es el jefe) y 'union' (hacerse amigos).
- Casi $O(1)$ gracias a Path Compression y Union by Rank.
- Ideal para detectar ciclos y conectividad en grafos.
- Solo funciona para conjuntos disjuntos (un elemento no puede estar en dos grupos a la vez).



Capítulo 19



Representación de Grafos



¿Qué es?

Un mapa de ciudades con carreteras entre ellas.

En computación, las ciudades se llaman Nodos (o Vértices) y las carreteras se llaman Aristas (o Edges). Un grafo es simplemente un conjunto de nodos conectados entre sí.

Puedes tener: - Grafos Dirigidos: Carreteras de un solo sentido ($A \rightarrow B$). - Grafos No Dirigidos: Carreteras de doble sentido ($A \leftrightarrow B$). - Grafos Ponderados: Carreteras con costo o distancia ($A \xrightarrow{5} B$).



Dicho de otra forma

Un Grafo $G = (V, E)$ es una estructura no lineal. Para trabajar con ellos en código, necesitamos guardarlos en memoria de alguna forma.

Las dos formas más comunes son: 1. Adjacency Matrix (Matriz de Adyacencia): Una tabla de $V \times V$. 2. Adjacency List (Lista de Adyacencia): Un array de listas.

La elección depende de si el grafo es "Denso" (muchas conexiones) o "Disperso" (pocas conexiones).



Diagrama

Grafo de ejemplo: A-B, A-C, B-C, C-D

Adjacency Matrix (4x4):

	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	1
D	0	0	1	0

Adjacency List:

A: [B, C]
B: [A, C]
C: [A, B, D]
D: [C]

- En la Matrix: 1 significa que hay conexión, 0 que no.
- En la List: Cada nodo tiene su propia lista de vecinos.

1. Adjacency Matrix:
 - Pros: Muy rápido $O(1)$ para saber si A y B están conectados.
 - Contras: Usa mucha memoria $O(V^2)$, incluso si hay pocos edges.
 - Cuando usar: Grafos pequeños o muy densos.

2. Adjacency List:

- Pros: Ahorra mucha memoria $O(V + E)$. Es la más usada.
- Contras: Saber si A y B están conectados puede tardar $O(V)$.
- Cuándo usar: Casi siempre (la mayoría de los grafos reales son dispersos).

PSEUDOCÓDIGO

```
// Agregar arista en Lista de Adyacencia (no dirigido)
```

```
función agregarArista(u, v):
```

```
    adj[u].agregar(v)
```

```
    adj[v].agregar(u)
```

```
// Agregar arista en Matriz de Adyacencia
```

```
función agregarAristaMatrix(u, v):
```

```
    matrix[u][v] = 1
```

```
    matrix[v][u] = 1
```

CÓDIGO C++

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    int V = 4; // Número de vértices
```

```
    // 1. Representación con Lista de Adyacencia (Recomendado)
```

```
    vector<vector<int>> adj(V);
```

```
    // Agregar A-B (0-1), A-C (0-2), B-C (1-2), C-D (2-3)
```

```
    adj[0].push_back(1); adj[1].push_back(0);
```

```
    adj[0].push_back(2); adj[2].push_back(0);
```

```
    adj[1].push_back(2); adj[2].push_back(1);
```

```
    adj[2].push_back(3); adj[3].push_back(2);
```

```
    cout << "Lista de Adyacencia de C (nodo 2): ";
```

```
    for(int vecino : adj[2]) {
```

```
        cout << vecino << " ";
```

```
    }
```

```
    cout << endl;
```

```
    // 2. Representación con Matriz de Adyacencia
```

```
    vector<vector<int>> matrix(V, vector<int>(V, 0));
```

```
    matrix[0][1] = 1; matrix[1][0] = 1;
```

```
    // ... completar resto
```

```
    return 0;
```

```
}
```

```

57
58 COMPLEJIDAD (Comparativa)
59
60
61 Operación      Matrix      List
62
63 Espacio         $O(V^2)$        $O(V + E)$ 
64 Agregar Edge    $O(1)$          $O(1)$ 
65 Verificar Edge  $O(1)$          $O(\text{Grado del nodo})$ 
66 Iterar Vecinos  $O(V)$          $O(\text{Grado del nodo})$ 
67
68 * V = Vértices, E = Edges (Aristas).

```

En entrevistas

Default: En el 90 % de las entrevistas, usa Adjacency List. Es más eficiente en memoria y espacio.

Representación con Map: A veces los nodos no son números (0, 1, 2) sino nombres ("CDMX", "NY"). En ese caso, usa: 'unordered_map<string, vector<string> adj;'

Edge List: Existe una tercera forma llamada "Edge List" que es simplemente un array de pares '[(0,1), (0,2), ...]'. Útil para algoritmos como Kruskal.

Grafo Implícito: A veces el grafo no te lo dan, sino que es una matriz (grid) donde puedes moverte arriba, abajo, izq, der. Ahí el "grafo" son las celdas y sus vecinos.

Resumen rápido

- Nodos = Ciudades, Aristas = Caminos.
- Matrix: Rápida pero traga mucha memoria $O(V^2)$.
- List: Eficiente en memoria, la favorita de los devs $O(V + E)$.
- Dirigido (flecha) vs No Dirigido (línea).
- Ponderado (tiene peso/costo) vs No Ponderado.
- Grado de un nodo = cuántos vecinos tiene.

Capítulo 20

🕒 BFS y DFS (Recorridos de Grafos)

🧐 ¿Qué es?

Dos formas de explorar un mundo desconocido :

1. BFS (Breadth-First Search): Como lanzar una piedra al agua . Ves las ondas expandirse nivel por nivel. Primero visitas a todos tus vecinos directos, luego a los vecinos de tus vecinos, y así sucesivamente. ¡No dejas a nadie atrás en tu nivel!
2. DFS (Depth-First Search): Como explorar un laberinto . Sigues un solo camino hasta el fondo, hasta que ya no puedas avanzar. Cuando llegas a un callejón sin salida, regresas (backtrack) y pruebas el siguiente camino disponible.

💬 Dicho de otra forma

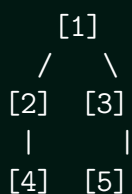
Son los dos algoritmos fundamentales para recorrer cualquier grafo o árbol.

- BFS usa una Queue (FIFO) y garantiza encontrar el camino más corto en grafos no ponderados. - DFS usa un Stack (LIFO) o Recursión. Es ideal para explorar todas las posibilidades o detectar ciclos.

En ambos casos, es CRUCIAL usar una estructura 'visited' (como un array de bools o un set) para no entrar en bucles infinitos.

🧠 Diagrama

Grafo:



BFS desde 1: 1 → 2,3 → 4,5

Orden: [1, 2, 3, 4, 5] (Nivel por nivel)

DFS desde 1: 1 → 2 → 4 → 5 → 3

Orden: [1, 2, 4, 5, 3] (Hasta el fondo)

```

1 BFS (con Queue):
2 1. Metes el nodo inicial a la Queue y lo marcas como visitado.
3 2. Mientras la Queue no esté vacía:
4   - Sacas el nodo del frente.
5   - Metes a todos sus vecinos NO visitados a la Queue y los marcas
6   .
7 DFS (con Recursión):
8 1. Marcas el nodo actual como visitado.
9 2. Para cada vecino NO visitado:
10  - Llamas a la función DFS recursivamente con ese vecino.
11
12 PSEUDOCÓDIGO
13
14
15 // BFS iterativo
16 función BFS(inicio):
17     q = crear Queue
18     q.enqueue(inicio)
19     marcar inicio como visitado
20     MIENTRAS q NO vacía:
21         u = q.dequeue()
22         PARA cada vecino v de u:
23             SI v NO visitado:
24                 q.enqueue(v)
25                 marcar v visitado
26
27 // DFS recursivo
28 función DFS(u):
29     marcar u como visitado
30     PARA cada vecino v de u:
31         SI v NO visitado:
32             DFS(v)
33
34 CÓDIGO C++
35
36
37 #include <iostream>
38 #include <vector>
39 #include <queue>
40 #include <stack>
41 using namespace std;
42
43 void bfs(int start, vector<vector<int>>& adj, vector<bool>& visited
44 ) {
45     queue<int> q;
46     q.push(start);
47     visited[start] = true;
48
49     while (!q.empty()) {
50         int u = q.front(); q.pop();

```

```

50     cout << u << " ";
51     for (int v : adj[u]) {
52         if (!visited[v]) {
53             visited[v] = true;
54             q.push(v);
55         }
56     }
57 }
58 }
59
60 void dfs(int u, vector<vector<int>>& adj, vector<bool>& visited) {
61     visited[u] = true;
62     cout << u << " ";
63     for (int v : adj[u]) {
64         if (!visited[v]) dfs(v, adj, visited);
65     }
66 }
67
68 int main() {
69     int V = 6;
70     vector<vector<int>> adj(V);
71     // Definir grafo...
72
73     vector<bool> visited(V, false);
74     cout << "Recorrido BFS: ";
75     bfs(0, adj, visited); // Suponiendo que empezamos en 0
76
77     fill(visited.begin(), visited.end(), false);
78     cout << "\nRecorrido DFS: ";
79     dfs(0, adj, visited);
80
81     return 0;
82 }
83
84 COMPLEJIDAD (Big-O)
85
86
87     Algoritmo         Tiempo         Espacio
88
89     BFS                O(V + E)        O(V)
90     DFS                O(V + E)        O(V)
91
92 * V = Vértices, E = Edges.
93 * El espacio O(V) es para la Queue/Stack y el array `visited`.

```



En entrevistas

BFS para Shortest Path: Si te piden el camino más corto en un grafo SIN pesos (o una cuadrícula), BFS es la única opción correcta.

DFS para Backtracking: Si el problema pide "encuentra todas las rutas posibles" o "prueba todas las combinaciones", DFS es tu mejor amigo.

Cycle Detection: - Undirected: Si encuentras un vecino visitado que no es tu padre. - Directed: Necesitas 3 estados (Blanco, Gris, Negro). Si ves un Gris, ¡hay ciclo!

Nivel de cada nodo: BFS es excelente para guardar la distancia (nivel) de cada nodo respecto al origen.



Resumen rápido

- BFS = Queue = Nivel por nivel = Camino más corto.
- DFS = Stack/Recursión = Hasta el fondo = Exploración total.
- ¡Siempre usa un array 'visited'!
- Ambos tardan $O(V + E)$.
- Úsalos en matrices (grid), redes sociales, mapas y juegos.

Capítulo 21

BFS Avanzado (Grid, Multi-source y 0-1 BFS)

¿Qué es?

El BFS normal explora un mapa nivel por nivel. Estas variantes son el mismo BFS con pequeños giros para problemas muy comunes en entrevistas:

- Grid: el "grafo" es una cuadrícula (un laberinto o mapa). Cada celda se conecta con sus 4 vecinos (arriba, abajo, izquierda, derecha).
- Multi-source: en vez de empezar en un punto, empiezas en VARIOS a la vez. Como varios incendios propagándose simultáneamente.
- 0-1 BFS: aristas que cuestan solo 0 o 1. Con una doble cola (deque) obtienes el camino más corto sin necesitar Dijkstra.

Dicho de otra forma

Muchos problemas de "camino más corto" no vienen como grafos explícitos, sino como matrices. Tratar cada celda como un nodo y sus 4 (u 8) vecinos como aristas convierte el problema en un BFS estándar.

Multi-source BFS: metes todos los orígenes en la Queue al inicio (distancia 0). El BFS expande todos los frentes en paralelo, dando la distancia mínima a CUALQUIER origen. Evita correr BFS por cada fuente.

0-1 BFS: cuando los pesos son solo 0 o 1, una deque sustituye a la cola de prioridad de Dijkstra: las aristas de peso 0 van al FRENTE y las de peso 1 al FINAL. Resultado: $O(V + E)$ en vez de $O(E \log V)$.

Diagrama

GRID (4 direcciones)

```
. . . #  
. # . .  
. . . .  
# . . .
```

vecinos de
(r,c):
(r+1,c) (r-1,c)
(r,c+1) (r,c-1)

MULTI-SOURCE (2 incendios)

```
Inicio: F en (0,0) y (2,3)  
F . . . dist=0 en ambos  
. . . F y crece hacia  
. . . . afuera a la vez.
```

dr[] = {1,-1,0,0} dc[] = {0,0,1,-1}

Multi-source: encola TODAS las fuentes con dist=0 al inicio.

```

1 BFS EN GRID
2 - Cada celda (r,c) es un nodo. Vecinos válidos: dentro de límites y
3   no bloqueados. Usa arreglos de direcciones dr[]/dc[].
4
5 MULTI-SOURCE BFS
6 - Encola TODAS las fuentes con distancia 0 antes del bucle. El
7   resto
8   del BFS es idéntico.
9
10 PSEUDOCÓDIGO (multi-source en grid)
11
12 función multiSourceBFS(grid, fuentes):
13     dist = matriz de -1 (no visitado)
14     q = Queue
15     PARA cada fuente (r,c): dist[r][c]=0; q.push((r,c))
16     MIENTRAS q no vacia:
17         (r,c) = q.pop()
18         PARA cada direccion (dr,dc):
19             nr=r+dr; nc=c+dc
20             SI dentro de limites Y grid libre Y dist[nr][nc]==-1:
21                 dist[nr][nc] = dist[r][c]+1
22                 q.push((nr,nc))
23     return dist
24
25 CÓDIGO C++ (BFS en grid)
26
27
28 #include <vector>
29 #include <queue>
30 using namespace std;
31
32 int dr[] = {1, -1, 0, 0};
33 int dc[] = {0, 0, 1, -1};
34
35 vector<vector<int>>> bfsGrid(vector<vector<int>>& grid,
36                             vector<pair<int,int>>& sources) {
37     int R = grid.size(), C = grid[0].size();
38     vector<vector<int>>> dist(R, vector<int>(C, -1));
39     queue<pair<int,int>>> q;
40     for (auto [r, c] : sources) { dist[r][c] = 0; q.push({r, c}); }
41
42     while (!q.empty()) {
43         auto [r, c] = q.front(); q.pop();
44         for (int d = 0; d < 4; d++) {
45             int nr = r + dr[d], nc = c + dc[d];
46             if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
47             if (grid[nr][nc] == 1 || dist[nr][nc] != -1) continue;
              // muro/visto

```

```

48         dist[nr][nc] = dist[r][c] + 1;
49         q.push({nr, nc});
50     }
51 }
52 return dist;
53 }
54
55 CÓDIGO C++ (0-1 BFS con deque)
56
57
58 #include <deque>
59 // adj[u] = lista de {vecino, peso} con peso 0 o 1
60 vector<int> zeroOneBFS(int V, vector<vector<pair<int,int>>>& adj,
61     int src) {
62     vector<int> dist(V, INT_MAX);
63     deque<int> dq;
64     dist[src] = 0; dq.push_front(src);
65     while (!dq.empty()) {
66         int u = dq.front(); dq.pop_front();
67         for (auto [v, w] : adj[u]) {
68             if (dist[u] + w < dist[v]) {
69                 dist[v] = dist[u] + w;
70                 if (w == 0) dq.push_front(v); // peso 0 -> al
71                     // frente
72                 else dq.push_back(v); // peso 1 -> al
73                     // final
74             }
75         }
76     }
77     return dist;
78 }
79
80 COMPLEJIDAD (Big-O)
81
82
83
84

```

Variante	Tiempo	Nota
BFS en grid	$O(R * C)$	R filas, C columnas
Multi-source	$O(R * C)$	igual, varios orígenes
0-1 BFS	$O(V + E)$	reemplaza a Dijkstra

En entrevistas

”Rotting Oranges” (LeetCode 994): multi-source BFS puro. Todas las naranjas podridas son fuentes; el resultado es el tiempo hasta podrir todo.

”01 Matrix” (LeetCode 542): distancia de cada celda al 0 más cercano = multi-source BFS desde todos los ceros a la vez.

8 direcciones vs 4: lee bien el enunciado. Si permiten diagonales, agrega las 4 direcciones diagonales a `dr[]/dc[]`.

0-1 BFS gana a Dijkstra cuando los pesos son binarios: mismo resultado, más rápido y sin heap. Buen detalle para destacar.

Resumen rápido

- Grid = grafo implícito: cada celda es un nodo, 4 u 8 vecinos.
- Usa `dr[]/dc[]` para recorrer direcciones limpiamente.
- Multi-source: encola TODAS las fuentes con dist 0 al inicio.
- 0-1 BFS: deque, peso 0 al frente, peso 1 al final; $O(V+E)$.
- Clásicos: Rotting Oranges, 01 Matrix, laberintos.

Capítulo 22

Algoritmo de Dijkstra (Camino más Corto)

¿Qué es?

El GPS de tu teléfono . Cuando pides la ruta más rápida de tu casa al trabajo, el GPS no intenta probar absolutamente todos los caminos posibles al azar.

En su lugar, siempre expande primero el camino que parece más prometedor (el más corto que ha encontrado hasta ahora). Va construyendo la ruta óptima paso a paso, actualizando sus cálculos si encuentra un "atajo" mejor.

Es un algoritmo "Greedy" (Codicioso) que encuentra el camino más corto desde un nodo origen a todos los demás nodos en un grafo con pesos.

Dicho de otra forma

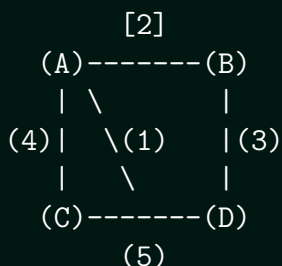
Dijkstra resuelve el problema del "Single-Source Shortest Path" en un grafo ponderado (weighted graph).

Funciona manteniendo una lista de distancias tentativas y usando un Min-Heap (Priority Queue) para elegir siempre el nodo con la distancia mínima actual.

Restricción Vital: NO funciona si el grafo tiene aristas con PESOS NEGATIVOS. (Para eso se usa Bellman-Ford).

Diagrama

Grafo Ponderado:



Distancias desde A:

1. Inicio: {A:0, B:∞, C:∞, D:∞}
2. Visitar A: Vecinos B(2), C(4), D(1).
Heap elige D(1).

3. Visitar D: Vecinos B(1+3=4), C(1+5=6).
 $\text{¿}4 < \text{dist}[B]$? No. $\text{¿}6 < \text{dist}[C]$? No.
4. Heap elige B(2).
5. Heap elige C(4).

Final: {A:0, B:2, C:4, D:1}

1. Inicializas todas las distancias como Infinito $\infty()$, excepto el nodo origen que es 0.
2. Metes (distancia 0, origen) a una Priority Queue (Min-Heap).
3. Mientras la Queue no esté vacía:
 - Sacas el nodo `u` con la menor distancia.
 - Si la distancia sacada es mayor a la que ya conocemos, ignora.
 - Para cada vecino `v` de `u`:
 - Calculas `nueva_dist = dist[u] + peso(u, v)`.
 - SI `nueva_dist < dist[v]`:
 - Actualizas `dist[v] = nueva_dist`.
 - Metes `(nueva_dist, v)` a la Priority Queue.

PSEUDOCÓDIGO

```

función Dijkstra(grafo, origen):
    distancias = array de tamaño V con INFINITO
    distancias[origen] = 0
    pq = crear Min-Heap de pares (distancia, nodo)
    pq.push(0, origen)

    MIENTRAS pq NO vacía:
        (d, u) = pq.pop_min()
        SI d > distancias[u]: continuar

        PARA cada vecino v de u con peso w:
            SI distancias[u] + w < distancias[v]:
                distancias[v] = distancias[u] + w
                pq.push(distancias[v], v)
  
```

CÓDIGO C++

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;

const int INF = 1e9;

void dijkstra(int start, vector<vector<pair<int, int>>>& adj) {
    int V = adj.size();
    vector<int> dist(V, INF);
    // Min-heap: priority_queue<T, container, comparison>
  
```

```

45     priority_queue<pair<int, int>, vector<pair<int, int>>, greater<
46         pair<int, int>>> pq;
47
48     dist[start] = 0;
49     pq.push({0, start});
50
51     while (!pq.empty()) {
52         int d = pq.top().first;
53         int u = pq.top().second;
54         pq.pop();
55
56         if (d > dist[u]) continue;
57
58         for (auto& edge : adj[u]) {
59             int v = edge.first;
60             int weight = edge.second;
61
62             if (dist[u] + weight < dist[v]) {
63                 dist[v] = dist[u] + weight;
64                 pq.push({dist[v], v});
65             }
66         }
67
68         for(int i = 0; i < V; i++)
69             cout << "Distancia a " << i << ": " << dist[i] << endl;
70     }
71
72     int main() {
73         int V = 4;
74         vector<vector<pair<int, int>>> adj(V);
75         // Definir grafo con pesos...
76         dijkstra(0, adj);
77         return 0;
78     }

```

COMPLEJIDAD (Big-O)

Operación	Tiempo	Razón
Dijkstra	$O((V + E) \log V)$	Priority Queue
Espacio	$O(V + E)$	Grafo + distancias

* V = Vértices, E = Aristas.

En entrevistas

Cuándo usar: "Shortest Path" en grafos con PESOS positivos. Si no hay pesos, usa BFS (es más rápido $O(V+E)$).

Pesos Negativos: Si el entrevistador pregunta "¿Y si hay pesos negativos?", responde: "Dijkstra fallaría, usaría Bellman-Ford".

C++ Priority Queue: Por defecto es un MAX-heap. Para Dijkstra necesitas un MIN-heap, así que usa 'greater<>' o guarda las distancias negativas.

Variante: "Cheapest Flights Within K Stops" es un problema común de Google que modifica Dijkstra limitando la profundidad.

Resumen rápido

- Algoritmo Codicioso (Greedy).
- Usa un Min-Heap para elegir siempre el camino más corto actual.
- Encuentra distancias mínimas desde UN origen a TODOS los demás.
- No admite pesos negativos.
- Tiempo: $O(E \log V)$.
- Es la base de los sistemas de navegación modernos.

Capítulo 23

Bellman-Ford (Camino más Corto con Pesos Negativos)

¿Qué es?

Buscar la ruta más barata entre ciudades, pero donde algunas carreteras te PAGAN por pasar (peso negativo), por ejemplo un descuento o un reembolso.

Dijkstra se rompe con esos reembolsos porque asume que avanzar siempre "cuesta". Bellman-Ford no asume nada: repite el cálculo muchas veces, relajando todas las carreteras una y otra vez, hasta que ninguna ruta puede mejorar más.

Además detecta las trampas: si existe un bucle que te paga infinito (ciclo negativo), Bellman-Ford lo denuncia.

Dicho de otra forma

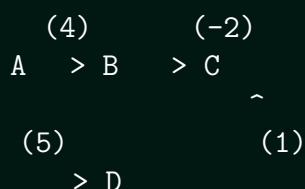
Bellman-Ford calcula la distancia más corta desde un nodo origen a todos los demás en un grafo dirigido y ponderado, admitiendo pesos negativos (algo que Dijkstra no permite).

Su idea central es la "relajación" (relaxation): recorrer todas las aristas $V - 1$ veces, actualizando $\text{dist}[v]$ cada vez que $\text{dist}[u] + \text{peso}(u,v) < \text{dist}[v]$. Tras $V - 1$ pasadas, las distancias son óptimas si no hay ciclos negativos.

Una pasada extra (V -ésima): si alguna arista todavía se puede relajar, existe un ciclo de peso negativo alcanzable desde el origen.

Diagrama

Grafo dirigido con peso negativo:



dist inicial: $A=0$, $B=\text{inf}$, $C=\text{inf}$, $D=\text{inf}$

Relajar aristas $V-1 = 3$ veces:

$A \rightarrow B(4)$: $B = 4$

```

B->C(-2): C = 2
A->D(5): D = 5
D->C(1): C = min(2, 6) = 2

```

Resultado: A=0, B=4, C=2, D=5

Pasada extra: ninguna arista mejora -> NO hay ciclo negativo.

```

1  CÓMO FUNCIONA
2  1. dist[origen] = 0, el resto = infinito.
3  2. Repite V-1 veces: por CADA arista (u, v, peso), relaja:
4     si dist[u] + peso < dist[v], entonces dist[v] = dist[u] + peso.
5  3. Pasada V-ésima: si alguna arista aún relaja -> ciclo negativo.
6
7  PSEUDOCÓDIGO
8
9
10 función bellmanFord(V, aristas, origen):
11     dist = arreglo lleno de INFINITO
12     dist[origen] = 0
13     REPETIR V-1 veces:
14         PARA cada (u, v, w) en aristas:
15             SI dist[u] != INF Y dist[u] + w < dist[v]:
16                 dist[v] = dist[u] + w
17     // Deteccion de ciclo negativo
18     PARA cada (u, v, w) en aristas:
19         SI dist[u] != INF Y dist[u] + w < dist[v]:
20             return "CICLO NEGATIVO"
21     return dist
22
23 CÓDIGO C++
24
25
26 #include <iostream>
27 #include <vector>
28 #include <climits>
29 using namespace std;
30
31 struct Edge { int u, v, w; };
32
33 bool bellmanFord(int V, vector<Edge>& edges, int src, vector<long
34 long>& dist) {
35     dist.assign(V, LLONG_MAX);
36     dist[src] = 0;
37
38     // V-1 pasadas de relajación
39     for (int i = 0; i < V - 1; i++) {
40         for (auto& e : edges) {
41             if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.
42                 v])

```

```

41         dist[e.v] = dist[e.u] + e.w;
42     }
43 }
44 // Pasada extra: si algo relaja, hay ciclo negativo
45 for (auto& e : edges) {
46     if (dist[e.u] != LLONG_MAX && dist[e.u] + e.w < dist[e.v])
47         return false; // ciclo negativo detectado
48 }
49 return true;
50 }
51
52 COMPLEJIDAD (Big-O)
53
54
55 Caso            Tiempo            Espacio
56
57 Tiempo          O(V * E)          O(V)
58
59 * Más lento que Dijkstra (O(E log V)) pero admite pesos negativos.

```

```

1
2 Optimización: si en una pasada completa NO se relajó ninguna
   arista,
3   ya terminaste antes de tiempo (early exit).
4
5 SPFA: una variante con Queue (Shortest Path Faster Algorithm) que
6   suele ser más rápida en la práctica, aunque su peor caso sigue
7   siendo O(V*E).
8
9 Aristas negativas SÍ, ciclos negativos NO: un peso negativo
   aislado
10  está bien; lo que rompe todo es un CICLO cuya suma sea negativa.

```



En entrevistas

¿Cuándo elegirlo sobre Dijkstra? Cuando haya pesos negativos o te pidan DETECTAR ciclos negativos. Es la respuesta esperada.

”Cheapest Flights Within K Stops” (LeetCode 787): variante de Bellman-Ford limitando el número de pasadas a $K+1$.

Inicializa con cuidado: nunca relajes desde un nodo con distancia infinita (evita overflow). Por eso el chequeo `dist[u] != INF`.

Frase clave: ”V-1 relajaciones porque el camino más corto tiene a lo sumo V-1 aristas”. Dila y sumas puntos.



Resumen rápido

- Camino más corto de una fuente a todos, CON pesos negativos.
- Relaja TODAS las aristas V-1 veces.
- Pasada extra detecta ciclos de peso negativo.
-

$O(V * E)$: más lento que Dijkstra pero más general. • Usa Dijkstra si todos los pesos son positivos (más rápido). • Clásico: rutas con reembolsos, arbitraje de divisas.

Capítulo 24

🌐 Floyd-Warshall (Caminos más Cortos entre Todos los Pares)

🧐 ¿Qué es?

Una tabla de distancias de un mapa de carreteras, donde cada casilla te dice cuánto cuesta ir de la ciudad X a la ciudad Y.

Floyd-Warshall llena esa tabla probando algo simple para cada par: "¿me conviene ir directo de X a Y, o es más barato pasar por una ciudad intermedia K?". Prueba TODAS las ciudades intermedias posibles, una por una, y se queda con la ruta más barata.

Al terminar, tienes la distancia mínima entre cualquier par de ciudades.

💬 Dicho de otra forma

Floyd-Warshall calcula la distancia más corta entre TODOS los pares de nodos de un grafo dirigido y ponderado. Admite pesos negativos (pero no ciclos negativos).

Es programación dinámica pura: para cada nodo intermedio k , mejora la distancia de todo par (i, j) preguntando si pasar por k es más corto:

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

donde d es la matriz de distancias.

Su elegancia es un triple bucle de 3 líneas. Su precio es $O(V^3)$ en tiempo y $O(V^2)$ en memoria, así que solo sirve para grafos pequeños o densos (cientos de nodos, no millones).

🧠 Diagrama

Matriz de distancias (inf = sin conexión directa):

	A	B	C
A	[0	3	inf]
B	[inf	0	1]
C	[2	inf	0]

Probando intermedio $k = B$:

A->C ahora puede pasar por B: A->B(3) + B->C(1) = 4

```
dist[A][C] = min(inf, 4) = 4
```

Probando intermedio $k = C$:

$B \rightarrow A$ puede pasar por C : $B \rightarrow C(1) + C \rightarrow A(2) = 3$

```
dist[B][A] = min(inf, 3) = 3
```

Matriz final (todas las distancias mínimas):

	A	B	C
A	[0	3	4]
B	[3	0	1]
C	[2	5	0]

CÓMO FUNCIONA

1. $dist[i][j]$ = peso de la arista $i \rightarrow j$ (0 si $i=j$, inf si no hay arista).
2. Por cada nodo intermedio k (bucle externo):
por cada par (i, j) :
 $dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j])$
3. ¡El orden importa! k DEBE ser el bucle más externo.
4. Si algún $dist[i][i]$ queda negativo $\rightarrow$ hay ciclo negativo.

PSEUDOCÓDIGO

```
función floydWarshall(dist):    // dist es matriz V x V
```

```
    PARA k desde 0 hasta V-1:
```

```
        PARA i desde 0 hasta V-1:
```

```
            PARA j desde 0 hasta V-1:
```

```
                SI  $dist[i][k] + dist[k][j] < dist[i][j]$ :
```

```
                     $dist[i][j] = dist[i][k] + dist[k][j]$ 
```

```
    return dist
```

CÓDIGO C++

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
const long long INF = 1e18;
```

```
void floydWarshall(vector<vector<long long>>& dist) {
```

```
    int V = dist.size();
```

```
    for (int k = 0; k < V; k++)
```

```
        for (int i = 0; i < V; i++)
```

```
            for (int j = 0; j < V; j++)
```

```
                if ( $dist[i][k] < INF \ \&\& \ dist[k][j] < INF \ \&\&$ 
```

```
                     $dist[i][k] + dist[k][j] < dist[i][j]$ )
```

```
                     $dist[i][j] = dist[i][k] + dist[k][j];$ 
```

```
}
```

```

38
39 int main() {
40     int V = 3;
41     vector<vector<long long>> dist(V, vector<long long>(V, INF));
42     for (int i = 0; i < V; i++) dist[i][i] = 0;
43     dist[0][1] = 3; dist[1][2] = 1; dist[2][0] = 2; // aristas
44     floydWarshall(dist);
45     // dist[i][j] ahora tiene la distancia mínima entre cada par
46     return 0;
47 }
48
49 COMPLEJIDAD (Big-O)
50
51
52 Recurso      Costo
53
54 Tiempo       $O(V^3)$ 
55 Espacio      $O(V^2)$ 
56
57 * Ideal para V pequeño (400-500). Para V grande usa Dijkstra por
    nodo.

```



En entrevistas

¿Cuándo usarlo? Cuando pidan distancias entre TODOS los pares y el grafo sea pequeño/denso. Si solo necesitas de UNA fuente, usa Dijkstra o Bellman-Ford.

Cierre transitivo: cambia (min, +) por (OR, AND) y Floyd-Warshall te dice qué nodos son alcanzables desde cuáles. Mismo esqueleto.

El orden de los bucles NO se negocia: k SIEMPRE va afuera. Si lo pones adentro, el resultado es incorrecto.

Ciclo negativo: revisa la diagonal. Si algún $\text{dist}[i][i] < 0$, existe un ciclo de peso negativo pasando por i.



Resumen rápido

- Distancias más cortas entre TODOS los pares de nodos.
- Programación dinámica: intermedio k mejora cada par (i, j).
- Triple bucle con k por fuera; $O(V^3)$ tiempo, $O(V^2)$ espacio.
- Admite pesos negativos, detecta ciclos negativos en la diagonal.
- Solo para grafos pequeños o densos.
- Alternativa: Dijkstra por cada nodo si el grafo es disperso.

Capítulo 25

Topological Sort (Ordenamiento Topológico)

¿Qué es?

Ponerte la ropa en el orden correcto . No puedes ponerte los zapatos antes que los calcetines, ni el saco antes que la camisa.

Cada prenda "depende" de otra. El Ordenamiento Topológico te da una lista válida de en qué orden vestirtte para que nunca rompas una regla: "primero A, luego B".

Solo funciona si NO hay contradicciones (ciclos). Si la regla dijera "ponte los zapatos antes que los calcetines Y los calcetines antes que los zapatos", sería imposible: eso es un ciclo.

Dicho de otra forma

El Topological Sort es un ordenamiento lineal de los vértices de un grafo dirigido acíclico (DAG) tal que, para toda arista $u \rightarrow v$, el nodo u aparece antes que v en el orden.

Solo existe en grafos dirigidos y SIN ciclos (DAG). Si hay un ciclo, no hay orden válido. Se usa para modelar dependencias: compilación de módulos, prerequisites de materias, planificación de tareas y resolución de fórmulas en hojas de cálculo.

Hay dos formas de calcularlo: el algoritmo de Kahn (BFS con in-degree) y el enfoque DFS (post-orden invertido).

Diagrama

Grafo de dependencias (DAG):

```
[Calcetines] --> [Zapatos]
[Camisa] --> [Saco] --> [Corbata]
[Ropa interior] --> [Pantalon] --> [Zapatos]
```

In-degree (aristas entrantes):

```
Calcetines:0  Camisa:0  RopaInterior:0
Pantalon:1   Zapatos:2  Saco:1   Corbata:1
```

Un orden topologico valido:

RopaInterior -> Pantalón -> Calcetines -> Zapatos
-> Camisa -> Saco -> Corbata

(No es unico: cualquier orden que respete las flechas sirve).

```

1 ALGORITMO DE KAHN (BFS con grados de entrada)
2 1. Calcula el in-degree (aristas entrantes) de cada nodo.
3 2. Mete a una Queue todos los nodos con in-degree = 0.
4 3. Mientras la Queue no esté vacía:
5   - Sacar un nodo u, agrégalo al resultado.
6   - Por cada vecino v de u: resta 1 a su in-degree.
7     Si su in-degree llega a 0, mételo a la Queue.
8 4. Si el resultado tiene menos nodos que V -> HAY CICLO (no es DAG)

```

PSEUDOCÓDIGO (Kahn)

```

13 función topoSort(V, adj):
14     indeg = arreglo de ceros de tamaño V
15     PARA cada u, PARA cada v en adj[u]: indeg[v]++
16     q = Queue con todos los nodos donde indeg == 0
17     orden = []
18     MIENTRAS q NO vacía:
19         u = q.dequeue(); orden.push(u)
20         PARA cada vecino v de u:
21             indeg[v]--
22             SI indeg[v] == 0: q.enqueue(v)
23     SI orden.size < V: return "CICLO DETECTADO"
24     return orden

```

CÓDIGO C++ (Kahn)

```

29 #include <iostream>
30 #include <vector>
31 #include <queue>
32 using namespace std;
33
34 vector<int> topoSort(int V, vector<vector<int>>& adj) {
35     vector<int> indeg(V, 0);
36     for (int u = 0; u < V; u++)
37         for (int v : adj[u]) indeg[v]++;
38
39     queue<int> q;
40     for (int i = 0; i < V; i++)
41         if (indeg[i] == 0) q.push(i);
42
43     vector<int> orden;
44     while (!q.empty()) {

```

```

45     int u = q.front(); q.pop();
46     orden.push_back(u);
47     for (int v : adj[u])
48         if (--indeg[v] == 0) q.push(v);
49 }
50 // Si no salieron todos, había un ciclo
51 if ((int)orden.size() != V) return {}; // vacío = no es DAG
52 return orden;
53 }
54
55 CÓDIGO C++ (DFS, post-orden invertido)
56
57
58 void dfs(int u, vector<vector<int>>& adj,
59         vector<bool>& vis, vector<int>& pila) {
60     vis[u] = true;
61     for (int v : adj[u])
62         if (!vis[v]) dfs(v, adj, vis, pila);
63     pila.push_back(u); // se agrega al TERMINAR
64 }
65 // El orden topológico es 'pila' invertida.
66
67 COMPLEJIDAD (Big-O)
68
69
70 Método      Tiempo      Espacio
71
72 Kahn (BFS)   O(V + E)      O(V)
73 DFS         O(V + E)      O(V)
74
75 * V = Vértices, E = Aristas.

```

En entrevistas

Detección de ciclos gratis: Kahn te dice si hay ciclo sin esfuerzo extra: si procesas menos de V nodos, el grafo NO es un DAG.

”Course Schedule” (LeetCode 207/210): el problema estrella. ”¿Puedes tomar todas las materias dado un mapa de prerrequisitos?” = ¿existe orden topológico? Respuesta directa con Kahn.

Solo en DAGs dirigidos: si el grafo es no dirigido o tiene ciclos, el ordenamiento topológico NO existe. Menciónalo siempre.

Kahn vs DFS: Kahn es iterativo (sin riesgo de stack overflow) y detecta ciclos naturalmente. DFS es más corto pero recuerda invertir el post-orden al final.



Resumen rápido

- Ordena un DAG respetando dependencias: u antes que v si $u \rightarrow v$.
- Solo existe en grafos dirigidos SIN ciclos.
- Kahn: BFS con in-degree = 0; detecta ciclos si sobran nodos.
- DFS: apila en post-orden e invierte el resultado.
- Ambos $O(V + E)$.
- Usos: prerequisites, build systems, planificación de tareas.

Capítulo 26

MST: Kruskal y Prim (Árbol de Expansión Mínima)

¿Qué es?

Conectar todas las casas de un pueblo con cable de internet gastando lo MENOS posible. No importa la ruta exacta; solo que TODAS queden conectadas y el cable total sea el más barato.

Dos estrategias para lograrlo:

- Kruskal: ordena todos los cables del más barato al más caro y ve agregándolos, saltándote cualquiera que forme un círculo inútil.
- Prim: empieza en una casa y va "creciendo" la red, agregando siempre el cable más barato que toque una casa nueva.

Ambas llegan al mismo costo mínimo por caminos distintos.

Dicho de otra forma

Un Árbol de Expansión Mínima (Minimum Spanning Tree) de un grafo no dirigido y ponderado es un subconjunto de aristas que conecta TODOS los vértices, sin ciclos, con el peso total mínimo. Tiene exactamente $V - 1$ aristas.

Kruskal es "goloso por aristas": ordena las aristas por peso y las agrega si no crean ciclo. Para saber si crean ciclo usa Union-Find (DSU). Ideal para grafos dispersos.

Prim es "goloso por nodos": crece un solo árbol desde un nodo, eligiendo con una min-heap la arista más barata hacia un nodo fuera del árbol. Ideal para grafos densos.

Diagrama

Grafo no dirigido con pesos:

	1		3
A	B		C
	2		4
5			6
D	E		
	7		

Aristas ordenadas: A-B(1), A-E(2), B-C(3), B-E(4), A-D(5)...

Kruskal agrega (si no forma ciclo):

A-B(1) A-E(2) B-C(3)
 B-E(4) (A,B,E ya conectados -> ciclo)
 A-D(5)

MST = {A-B, A-E, B-C, A-D}, peso total = $1+2+3+5 = 11$
 (V=5 nodos -> $V-1 = 4$ aristas)

1 KRUSKAL (goloso por aristas + Union-Find)

2 1. Ordena todas las aristas por peso ascendente.

3 2. Recorre las aristas: si u y v están en componentes distintas
 4 (find(u) != find(v)), agrégala al MST y hazles **union()**.

5 3. Para cuando el MST tenga V-1 aristas.

6
 7 PSEUDOCÓDIGO (Kruskal)

8
 9
 10 función kruskal(V, aristas):

11 ordenar aristas por peso

12 dsu = UnionFind(V)

13 mst = 0; usadas = 0

14 PARA cada (w, u, v) en aristas:

15 SI dsu.find(u) != dsu.find(v):

16 dsu.**union**(u, v)

17 mst += w; usadas++

18 SI usadas == V-1: romper

19 **return** mst

20
 21 CÓDIGO C++ (Kruskal)

22
 23
 24 **#include** <iostream>

25 **#include** <vector>

26 **#include** <algorithm>

27 **using namespace** std;

28
 29 **struct** DSU {

30 vector<int> p, r;

31 DSU(int n): p(n), r(n, 0) { **for** (int i=0;i<n;i++) p[i]=i; }

32 int find(int x){ **return** p[x]==x ? x : p[x]=find(p[x]); }

33 bool unite(int a,int b){

34 a=find(a); b=find(b);

35 **if**(a==b) **return** false;

36 **if**(r[a]<r[b]) swap(a,b);

37 p[b]=a; **if**(r[a]==r[b]) r[a]++;

38 **return** true;

39 }

40 };
 41

```

42 // aristas: {peso, u, v}
43 long long kruskal(int V, vector<array<int,3>>& edges) {
44     sort(edges.begin(), edges.end()); // ordena por peso (primer
        campo)
45     DSU dsu(V);
46     long long total = 0; int usadas = 0;
47     for (auto& e : edges) {
48         if (dsu.unite(e[1], e[2])) { // no forma ciclo
49             total += e[0];
50             if (++usadas == V - 1) break;
51         }
52     }
53     return total;
54 }

```

CÓDIGO C++ (Prim con min-heap)

```

59 #include <queue>
60 long long prim(int V, vector<vector<pair<int,int>>>& adj) {
61     // adj[u] = lista de {vecino, peso}
62     vector<bool> inMST(V, false);
63     priority_queue<pair<int,int>, vector<pair<int,int>>,
64         greater<>> pq; // min-heap {peso, nodo}
65     pq.push({0, 0});
66     long long total = 0;
67     while (!pq.empty()) {
68         auto [w, u] = pq.top(); pq.pop();
69         if (inMST[u]) continue;
70         inMST[u] = true; total += w;
71         for (auto [v, pw] : adj[u])
72             if (!inMST[v]) pq.push({pw, v});
73     }
74     return total;
75 }

```

COMPLEJIDAD (Big-O)

Algoritmo	Tiempo	Mejor para
Kruskal	$O(E \log E)$	Grafos dispersos
Prim (heap)	$O(E \log V)$	Grafos densos

En entrevistas

Kruskal reusa Union-Find: si ya dominas DSU, Kruskal es casi gratis. Es la razón por la que se enseñan juntos.

”Min Cost to Connect All Points” (LeetCode 1584): MST clásico sobre distan-

cias de Manhattan. Prim suele ganar por ser el grafo denso.

MST solo en grafos NO dirigidos y conexos. Si el grafo está desconectado no existe MST (existe un "bosque" de expansión mínima).

Corte (cut property): la arista más barata que cruza cualquier partición del grafo SIEMPRE está en algún MST. Es la justificación teórica de por qué el enfoque goloso funciona aquí.

Resumen rápido

- MST: conecta todos los nodos, sin ciclos, con peso mínimo ($V-1$ aristas).
- Kruskal: ordena aristas + Union-Find; $O(E \log E)$; grafos dispersos.
- Prim: crece un árbol con min-heap; $O(E \log V)$; grafos densos.
- Ambos son golosos y dan el mismo costo mínimo.
- Solo en grafos no dirigidos y conexos.
- Kruskal es la mejor excusa para practicar Union-Find.

Capítulo 27

Detección de Ciclos en Grafos

¿Qué es?

Caminar por una ciudad y de repente darte cuenta de que ya pasaste por esa esquina: diste una vuelta completa. Eso es un ciclo.

Detectarlo depende del tipo de calle:

- Calles de doble sentido (grafo no dirigido): hay ciclo si llegas a un lugar ya visitado que NO es de donde acabas de venir.
- Calles de un solo sentido (grafo dirigido): hay ciclo si vuelves a pisar un lugar que todavía está "en tu camino actual", no uno que ya terminaste de explorar.

Dicho de otra forma

Detectar ciclos es fundamental: un ciclo cambia por completo qué algoritmos aplican (por ejemplo, el ordenamiento topológico solo existe si NO hay ciclos).

En grafos **no dirigidos** usamos DFS guardando el "padre": si encontramos un vecino ya visitado distinto del padre, hay ciclo. Alternativa: Union-Find (si dos extremos de una arista ya están unidos, esa arista cierra un ciclo).

En grafos **dirigidos** usamos DFS con 3 estados (colores): Blanco (sin visitar), Gris (en la pila de recursión actual) y Negro (terminado). Si durante el DFS tocamos un nodo Gris, hay ciclo. El algoritmo de Kahn también sirve: si el topological sort no procesa todos los nodos, hay ciclo.

Diagrama

NO DIRIGIDO (padre):

A B
D C

DFS: A->B->C->D->A

Al ver A de nuevo (no es padre de D) => CICLO

DIRIGIDO (3 colores):

A > B > C
^

DFS desde A:

A(gris) B(gris) C(gris)

C apunta a A que esta GRIS
=> CICLO

Estados: Blanco=0 Gris=1 (en recursion) Negro=2 (listo)

```

1 NO DIRIGIDO (DFS con padre)
2 - Recorre con DFS. Al visitar un vecino v de u:
3   * Si v no visitado -> DFS(v, padre=u).
4   * Si v visitado y v != padre -> hay ciclo.
5
6 DIRIGIDO (DFS con 3 colores)
7 - color[u] = GRIS al entrar, NEGRO al salir.
8 - Si un vecino v es GRIS -> hay ciclo (arista hacia atrás).
9
10 PSEUDOCÓDIGO (dirigido, 3 colores)
11
12
13 función tieneCicloDirigido(u):
14   color[u] = GRIS
15   PARA cada vecino v de u:
16     SI color[v] == GRIS: return true           // ciclo
17     SI color[v] == BLANCO Y tieneCicloDirigido(v): return true
18   color[u] = NEGRO
19   return false
20
21 CÓDIGO C++ (no dirigido, con padre)
22
23
24 #include <vector>
25 using namespace std;
26
27 bool dfsUndirected(int u, int parent, vector<vector<int>>& adj,
28                   vector<bool>& vis) {
29   vis[u] = true;
30   for (int v : adj[u]) {
31     if (!vis[v]) {
32       if (dfsUndirected(v, u, adj, vis)) return true;
33     } else if (v != parent) {
34       return true;           // visitado y no es el padre
35     }
36   }
37   return false;
38 }
39
40 CÓDIGO C++ (dirigido, 3 colores)
41
42
43 bool dfsDirected(int u, vector<vector<int>>& adj, vector<int>&
44                  color) {
45   color[u] = 1;           // GRIS (en recursión)
46   for (int v : adj[u]) {
47     if (color[v] == 1) return true;           // ciclo
48     if (color[v] == 0 && dfsDirected(v, adj, color)) return
49       true;

```

```

48     }
49     color[u] = 2;                // NEGRO (terminado)
50     return false;
51 }
52
53 COMPLEJIDAD (Big-O)
54
55
56 Método                Tiempo                Espacio
57
58 DFS (ambos)            O(V + E)            O(V)
59 Union-Find (undir)     O(E (V))          O(V)
60
61 * = función inversa de Ackermann, prácticamente constante.

```

En entrevistas

Elige el método según el grafo: no dirigido -> padre o Union-Find; dirigido -> 3 colores o Kahn. Confundirlos es un error típico.

"Course Schedule" (LeetCode 207): "¿es posible terminar el plan?" = "¿el grafo de prerequisites NO tiene ciclos?". Ciclo dirigido.

El truco del "padre" solo aplica a NO dirigidos. En dirigidos un nodo NEGRO (ya terminado) no implica ciclo; solo un GRIS lo implica.

Detección de deadlocks: en sistemas operativos, un ciclo en el grafo de "espera de recursos" = interbloqueo. Buen dato para mencionar.

Resumen rápido

- No dirigido: DFS con padre, o Union-Find (arista que une lo ya unido).
- Dirigido: DFS con 3 colores; un vecino GRIS = ciclo.
- Alternativa dirigida: Kahn no procesa todos los nodos = ciclo.
- Todo en $O(V + E)$.
- Vital para topo-sort, detección de deadlocks y validar DAGs.

Capítulo 28

Componentes Fuertemente Conexas (SCC · Kosaraju)

¿Qué es?

Grupos de amigos en una red social donde "seguir" es de un solo sentido. Una Componente Fuertemente Conexa es un grupo donde TODOS pueden llegar a TODOS siguiendo flechas: desde cualquier persona del grupo puedes alcanzar a cualquier otra y regresar.

Kosaraju encuentra estos grupos con un truco astuto: recorre el grafo, luego INVIERTE todas las flechas y lo recorre de nuevo. Los nodos que siguen "atrapados juntos" incluso con las flechas al revés forman una componente fuertemente conexa.

Dicho de otra forma

En un grafo dirigido, una Componente Fuertemente Conexa (Strongly Connected Component) es un subconjunto máximo de nodos donde existe un camino dirigido entre cualquier par de ellos en ambos sentidos.

El algoritmo de Kosaraju las halla en dos pasadas de DFS:

1. DFS sobre el grafo original, apilando cada nodo cuando TERMINA (orden de finalización, como en topo-sort).
2. Transponer el grafo (invertir todas las aristas) y hacer DFS sacando nodos de la pila; cada árbol de este segundo DFS es una SCC.

Alternativa de una sola pasada: el algoritmo de Tarjan (usa índices de descubrimiento y low-link), más eficiente pero más difícil de recordar bajo presión.

Diagrama

Grafo dirigido:

A → B → C → D
 ^ ^

A ↔ B forman ciclo → SCC {A, B}
C ↔ D forman ciclo → SCC {C, D}

Paso 1 (DFS original): apila por orden de finalizacion

Paso 2 (grafo transpuesto): cada DFS-tree = una SCC

Resultado: {A, B} y {C, D}

Grafo condensado (cada SCC = 1 super-nodo) SIEMPRE es un DAG.

```

1 KOSARAJU (dos pasadas de DFS)
2 1. DFS en el grafo original; al terminar cada nodo, apílalo.
3 2. Construye el grafo transpuesto (invierte todas las aristas).
4 3. Saca nodos de la pila; por cada no visitado, un DFS en el
5   transpuesto marca una SCC completa.

```

PSEUDOCÓDIGO

```

10 función kosaraju(V, adj):
11     // Pasada 1: orden por finalizacion
12     PARA cada u no visitado: dfs1(u) -> apila u al terminar
13     adjT = transponer(adj)
14     // Pasada 2: sacar de la pila sobre el transpuesto
15     MIENTRAS pila no vacia:
16         u = pila.pop()
17         SI u no visitado_2:
18             dfs2(u, adjT)           // marca una SCC entera
19             sccCount++
20     return sccCount

```

CÓDIGO C++

```

25 #include <vector>
26 #include <stack>
27 using namespace std;
28
29 void dfs1(int u, vector<vector<int>>& adj, vector<bool>& vis,
30          stack<int>& st) {
31     vis[u] = true;
32     for (int v : adj[u]) if (!vis[v]) dfs1(v, adj, vis, st);
33     st.push(u);           // apila al TERMINAR
34 }
35
36 void dfs2(int u, vector<vector<int>>& adjT, vector<bool>& vis,
37          vector<int>& comp) {
38     vis[u] = true; comp.push_back(u);
39     for (int v : adjT[u]) if (!vis[v]) dfs2(v, adjT, vis, comp);
40 }
41
42 int kosaraju(int V, vector<vector<int>>& adj) {
43     vector<bool> vis(V, false);
44     stack<int> st;

```

```

45     for (int i = 0; i < V; i++) if (!vis[i]) dfs1(i, adj, vis, st);
46
47     // Transponer
48     vector<vector<int>> adjT(V);
49     for (int u = 0; u < V; u++)
50         for (int v : adj[u]) adjT[v].push_back(u);
51
52     fill(vis.begin(), vis.end(), false);
53     int scc = 0;
54     while (!st.empty()) {
55         int u = st.top(); st.pop();
56         if (!vis[u]) {
57             vector<int> comp;
58             dfs2(u, adjT, vis, comp); // una SCC
59             scc++;
60         }
61     }
62     return scc;
63 }

```

COMPLEJIDAD (Big-O)

Algoritmo	Tiempo	Espacio
Kosaraju	$O(V + E)$	$O(V + E)$
Tarjan	$O(V + E)$	$O(V)$

* Tarjan hace una sola pasada; Kosaraju hace dos pero es más intuitivo.



En entrevistas

Grafo condensado = DAG: si colapsas cada SCC en un solo nodo, el grafo resultante nunca tiene ciclos. Sirve para aplicar topo-sort encima. Frase que impresiona.

”Critical Connections” / puentes (LeetCode 1192): variantes de conectividad que se resuelven con low-link estilo Tarjan.

SCC es solo para grafos DIRIGIDOS. En no dirigidos el concepto equivalente es ”componentes conexas” simples (un DFS/BFS o Union-Find).

Recuerda el orden de Kosaraju: primero apilas por finalización en el grafo original, luego DFS en el TRANSPUESTO. Invertir ese orden lo rompe.



Resumen rápido

- SCC: grupo máximo de nodos mutuamente alcanzables (grafo dirigido).
- Kosaraju: DFS original (apilar al terminar) + DFS en el transpuesto.
- Cada árbol del segundo DFS es una SCC.
- Tarjan: una sola pasada con low-link (más

rápido, más difícil). • Ambos $O(V + E)$. • Condensar las SCC produce siempre un DAG.

Algoritmos de Ordenamiento

Capítulo 29

□ Ordenamientos Básicos (Bubble, Insertion, Selection)

🧐 ¿Qué es?

Tres formas manuales de ordenar cosas en la vida real:

1. Bubble Sort (Burbujas): Imagina burbujas subiendo en un refresco. Comparas dos elementos juntos; si el de la izquierda es mayor, lo cambias. Los números más grandes "burbujean" hacia el final en cada pasada.
2. Insertion Sort (Cartas): Como cuando ordenas cartas en tu mano. Tomas una carta nueva y la vas comparando con las que ya tienes ordenadas hasta que encuentras su lugar exacto y la "insertas".
3. Selection Sort (Selección): Buscas el objeto más pequeño de todo el grupo y lo pones al principio. Luego buscas el más pequeño de los que quedan y lo pones en segundo lugar, y así...

💬 Dicho de otra forma

Son algoritmos de ordenamiento $O(n^2)$. Aunque son lentos para grandes cantidades de datos, son fundamentales para entender la lógica de sorting.

- Bubble Sort: Se basa en intercambios adyacentes. - Insertion Sort: Divide el array en una parte ordenada y otra no. Es muy eficiente para arrays que ya están casi ordenados. - Selection Sort: Se basa en encontrar el mínimo en cada iteración.

🧠 Diagrama

Bubble Sort (Paso 1):

[64, 34, 25, 12] -> 64 > 34? SÍ -> [34, 64, 25, 12]

[34, 64, 25, 12] -> 64 > 25? SÍ -> [34, 25, 64, 12]

[34, 25, 64, 12] -> 64 > 12? SÍ -> [34, 25, 12, 64]* (64 ya está al final)

Insertion Sort (Paso 2 - ordenando el 25):

Mano: [34, 64] | Pendiente: [25, 12]

Tomo el 25. ¿25 < 64? SÍ. ¿25 < 34? SÍ.

Mano: [25, 34, 64] | Pendiente: [12]

Selection Sort (Paso 1):

[64, 34, 25, 12] -> Mínimo es 12.
 Intercambio 64 y 12 -> [12, 34, 25, 64]

1 Bubble:

- 2 - Compara pares (i, i+1).
- 3 - Swapea si están en orden incorrecto.
- 4 - Repite n-1 veces.

5
 6 Insertion:

- 7 - Empieza en el segundo elemento.
- 8 - Compáralo hacia atrás y muévelo hasta que sea mayor que el anterior.

9
 10 Selection:

- 11 - Encuentra el índice del valor mínimo en el rango [i...n].
- 12 - Intercambia `arr[i]` con `arr[min\_idx]`.

13
 14 PSEUDOCÓDIGO

15
 16
 17 *// Bubble Sort*

```
18 PARA i desde 0 hasta n-1:
19     PARA j desde 0 hasta n-i-2:
20         SI arr[j] > arr[j+1]:
21             intercambiar(arr[j], arr[j+1])
```

22
 23 *// Insertion Sort*

```
24 PARA i desde 1 hasta n-1:
25     clave = arr[i]
26     j = i - 1
27     MIENTRAS j >= 0 Y arr[j] > clave:
28         arr[j+1] = arr[j]
29         j--
30     arr[j+1] = clave
```

31
 32 CÓDIGO C++

```
33  

34  

35 #include <iostream>
36 #include <vector>
37 using namespace std;
38  

39 void bubbleSort(vector<int>& arr) {
40     int n = arr.size();
41     for (int i = 0; i < n-1; i++) {
42         for (int j = 0; j < n-i-1; j++) {
43             if (arr[j] > arr[j+1]) swap(arr[j], arr[j+1]);
44         }
45     }
46 }
47
```

```

48 void insertionSort(vector<int>& arr) {
49     int n = arr.size();
50     for (int i = 1; i < n; i++) {
51         int key = arr[i];
52         int j = i - 1;
53         while (j >= 0 && arr[j] > key) {
54             arr[j + 1] = arr[j];
55             j--;
56         }
57         arr[j + 1] = key;
58     }
59 }
60
61 int main() {
62     vector<int> data = {64, 34, 25, 12, 22};
63     insertionSort(data);
64     for(int x : data) cout << x << " "; // 12 22 25 34 64
65     return 0;
66 }

```

COMPLEJIDAD (Big-O)

Algoritmo	Best	Average	Worst	Stable?
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	SÍ
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	SÍ
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	NO

* Todos usan $O(1)$ de espacio extra (In-place).

1 Insertion Sort es el mejor para arrays "casi ordenados" o para
2 pocos elementos ($n < 20$).
3
4
5 Estabilidad (Stability): Significa que elementos con el mismo
6 valor
7 mantienen su orden relativo original. Bubble e Insertion son
8 estables; Selection NO.
9
10 Casi nunca usarás estos en producción (C++ usa `std::sort` que es
11 $O(n \log n)$), pero te los piden para evaluar tus bases.
12
13 Optimización Bubble: Puedes agregar un flag `swapped`. Si en una
pasada completa no hubo cambios, ¡el array ya está ordenado!



Resumen rápido

- Los 3 son $O(n^2)$ en el peor caso.
- Bubble: El más sencillo e ineficiente.
- Insertion: El mejor para casos pequeños o casi listos.
- Selection: Hace el mínimo de swaps posibles (útil si escribir en memoria es muy caro).
- In-place: No necesitan memoria extra.

Capítulo 30

☒ Merge Sort (Ordenamiento por Mezcla)

🧐 ¿Qué es?

Dividir un problema grande con un amigo . Tienes una pila gigante de cartas desordenadas. En lugar de ordenarlas tú solo, le das la mitad a tu amigo.

Cada uno ordena su mitad (y si son muchas, vuelven a dividir con alguien más). Al final, cuando cada quien tiene su pequeña parte ordenada, las fusionan (merge) comparando carta por carta para armar la pila final perfecta.

Es un algoritmo basado en la estrategia "Divide y Vencerás" (Divide and Conquer).

💬 Dicho de otra forma

Merge Sort es un algoritmo de ordenamiento recursivo, estable y de comparación. Su funcionamiento se basa en dos fases: 1. Divide: Parte el array recursivamente a la mitad hasta llegar a unidades de tamaño 1. 2. Conquer (Merge): Combina los subarrays ordenados para formar uno más grande, manteniendo el orden.

A diferencia de Quick Sort, Merge Sort GARANTIZA un tiempo de ejecución de $O(n \log n)$ incluso en el peor caso posible.

🔄 Diagrama

Array inicial: [38, 27, 43, 3]

Divide:

```
[38, 27]      [43, 3]
[38]  [27]    [43]  [3]
```

Merge (ordenando al combinar):

```
[27, 38]      [3, 43]
  \           /
  [3, 27, 38, 43]  <-- ¡Ordenado!
```

1. Si el array tiene tamaño 1 o 0, ya está ordenado (caso base).
2. Encuentras el punto medio: `mid = size / 2``.
3. Llamas a ``mergeSort`` para la mitad izquierda.
4. Llamas a ``mergeSort`` para la mitad derecha.

5. Funcionas (Merge) las dos mitades:

- Comparas los elementos de cada mitad uno por uno.
- Copias el menor al array original.
- Repites hasta terminar ambas mitades.

PSEUDOCÓDIGO

función mergeSort(arr):

SI arr.tamaño <= 1: **return** arr

mitad = arr.tamaño / 2

izq = mergeSort(arr[0...mitad])

der = mergeSort(arr[mitad...fin])

return merge(izq, der)

función merge(izq, der):

resultado = []

MIENTRAS izq Y der NO vacíos:

SI izq[0] <= der[0]:

agregar izq[0] a resultado, quitar de izq

SINO:

agregar der[0] a resultado, quitar de der

agregar lo que sobre de izq o der a resultado

return resultado

CÓDIGO C++

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void merge(vector<int>& arr, int l, int m, int r) {
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    vector<int> L(n1), R(n2);
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
```

```
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
```

```
    int i = 0, j = 0, k = l;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) arr[k++] = L[i++];
```

```
        else arr[k++] = R[j++];
```

```
    }
```

```
    while (i < n1) arr[k++] = L[i++];
```

```
    while (j < n2) arr[k++] = R[j++];
```

```
}
```

```

56 void mergeSort(vector<int>& arr, int l, int r) {
57     if (l >= r) return;
58     int m = l + (r - l) / 2;
59     mergeSort(arr, l, m);
60     mergeSort(arr, m + 1, r);
61     merge(arr, l, m, r);
62 }
63
64 int main() {
65     vector<int> data = {38, 27, 43, 3, 9, 82, 10};
66     mergeSort(data, 0, data.size() - 1);
67     for(int x : data) cout << x << " "; // 3 9 10 27 38 43 82
68     return 0;
69 }
70
71 COMPLEJIDAD (Big-O)
72
73
74     Caso           Tiempo           Espacio
75
76     Best             $O(n \log n)$          $O(n)$ 
77     Average          $O(n \log n)$          $O(n)$ 
78     Worst            $O(n \log n)$          $O(n)$ 
79
80 * El espacio  $O(n)$  es porque necesita arrays auxiliares para mezclar
    .

```

En entrevistas

"Sort a Linked List": Merge Sort es el algoritmo preferido para ordenar Linked Lists, porque no necesita acceso aleatorio.

Estabilidad: Merge Sort es un Stable Sort. Esto es vital si ordenas objetos por varios criterios (ej: ordenar personas por nombre y luego por edad).

Memoria: Su punto débil es el espacio $O(n)$. Si la memoria es muy limitada, podrías preferir Quick Sort o Heap Sort.

Merge Step: La lógica de la función 'merge' se usa en otros problemas como "Merge Two Sorted Lists" o "Merge Intervals".

Resumen rápido

- Divide y vencerás.
- Siempre tarda $O(n \log n)$, sin importar qué tan desordenado esté.
- Es un ordenamiento estable.
- Usa memoria extra $O(n)$.
- Ideal para datasets grandes o Linked Lists.
- Base para problemas de fusionar listas o intervalos.

Capítulo 31

⚡ Quick Sort (Ordenamiento Rápido)

🤖 ¿Qué es?

Elegir a un jugador de referencia en el equipo (el Pivot).

Una vez que eliges al Pivot (por ejemplo, el último de la fila), separas a todos los demás en dos grupos: 1. Los que son "peores" (número menor) van a la izquierda del Pivot. 2. Los que son "mejores" (número mayor) van a la derecha del Pivot.

Ahora el Pivot quedó en su lugar correcto para siempre. Luego, haces lo mismo con los dos grupos que quedaron a los lados, hasta que todos estén ordenados.

💬 Dicho de otra forma

Quick Sort es un algoritmo de ordenamiento por comparación, recursivo e In-place.

Se basa en la operación "Partition": elegir un elemento como Pivot y reorganizar el array de modo que los menores queden a la izquierda y los mayores a la derecha.

Aunque su peor caso es $O(n^2)$, en la práctica es mucho más rápido que Merge Sort debido a que tiene una excelente "Cache Locality" y no requiere memoria extra significativa.

🧠 Diagrama

Array: [3, 6, 8, 10, 1, 2, 5] (Pivot = 5)

Partition:

- Elementos < 5: [3, 1, 2]
- Pivot: 5
- Elementos > 5: [6, 8, 10]

Estado: [3, 1, 2] [5] [6, 8, 10]
(El 5 ya no se mueve más).

Se repite el proceso para [3, 1, 2] y para [6, 8, 10].

1. Caso base: Si el rango tiene 0 o 1 elementos, ya está ordenado.
2. Partition:

```

3   - Eliges un Pivot (el primero, el último, el medio o uno
      aleatorio).
4   - Reacomodas el array: elementos < Pivot a la izquierda, > a la
      der.
5   - Devuelves la posición final del Pivot.
6 3. Llamas recursivamente a Quick Sort para la mitad izquierda.
7 4. Llamas recursivamente a Quick Sort para la mitad derecha.

```

9 PSEUDOCÓDIGO

```

12 función quickSort(arr, inicio, fin):
13     SI inicio < fin:
14         p_idx = partition(arr, inicio, fin)
15         quickSort(arr, inicio, p_idx - 1)
16         quickSort(arr, p_idx + 1, fin)

```

```

18 función partition(arr, inicio, fin):
19     pivot = arr[fin]
20     i = inicio - 1
21     PARA j desde inicio hasta fin - 1:
22         SI arr[j] < pivot:
23             i++
24             intercambiar(arr[i], arr[j])
25     intercambiar(arr[i+1], arr[fin])
26     return i + 1

```

28 CÓDIGO C++

```

31 #include <iostream>
32 #include <vector>
33 using namespace std;
34
35 int partition(vector<int>& arr, int low, int high) {
36     int pivot = arr[high]; // Elegimos el último como pivot
37     int i = low - 1;       // Índice del elemento más pequeño
38
39     for (int j = low; j < high; j++) {
40         if (arr[j] < pivot) {
41             i++;
42             swap(arr[i], arr[j]);
43         }
44     }
45     swap(arr[i + 1], arr[high]);
46     return i + 1;
47 }
48
49 void quickSort(vector<int>& arr, int low, int high) {
50     if (low < high) {
51         int pi = partition(arr, low, high);

```

```

52     quickSort(arr, low, pi - 1);
53     quickSort(arr, pi + 1, high);
54 }
55 }
56
57 int main() {
58     vector<int> data = {10, 7, 8, 9, 1, 5};
59     quickSort(data, 0, data.size() - 1);
60     for(int x : data) cout << x << " "; // 1 5 7 8 9 10
61     return 0;
62 }

```

COMPLEJIDAD (Big-O)

Caso	Tiempo	Cuándo ocurre
Best	$O(n \log n)$	El pivot siempre parte a la mitad
Average	$O(n \log n)$	Pivot aleatorio / balanceado
Worst	$O(n^2)$	Array ya ordenado + pivot extremo
Espacio	$O(\log n)$	Stack de recursión

```

1
2 std::sort: En C++, `std::sort` usa "Introsort", que empieza con
3 Quick Sort y cambia a Heap Sort si la recursión es muy profunda.
4
5 Pivot Selection: Para evitar el peor caso  $O(n^2)$ , menciona que se
6 puede usar un Pivot aleatorio o la técnica "Median of Three".
7
8 No es estable: A diferencia de Merge Sort, Quick Sort NO garantiza
9 mantener el orden relativo de elementos iguales.
10
11 In-place: Su gran ventaja es que no necesita arrays extras, solo
12 memoria para las llamadas recursivas (el stack).

```



Resumen rápido

- El más rápido en la práctica.
- Usa la estrategia "Divide y Vencerás".
- In-place (ahorra memoria).
- Peor caso $O(n^2)$ pero muy raro con buen pivot.
- No es estable.
- Operación clave: 'partition'.

Algoritmos de Búsqueda

Capítulo 32

👁 Linear Search (Búsqueda Lineal)

🤖 ¿Qué es?

Buscar tus llaves en casa revisando cuarto por cuarto, cajón por cajón, sin ningún orden especial.

Empiezas en un lugar y revisas todo hasta que las encuentras o hasta que verificas que no están en ningún lado. Es la forma más básica y natural de buscar algo.

💬 Dicho de otra forma

Linear Search es un algoritmo de búsqueda que recorre un conjunto de datos secuencialmente (uno por uno) de principio a fin.

No requiere que los datos estén ordenados. Su simplicidad lo hace ideal para datasets pequeños o cuando solo vas a buscar algo una vez y no vale la pena ordenar todo el conjunto primero.

🧠 Diagrama

Buscar el número 7 en el array:
[3, 1, 9, 7, 5]

Paso 1: ¿3 == 7? No.

Paso 2: ¿1 == 7? No.

Paso 3: ¿9 == 7? No.

Paso 4: ¿7 == 7? ¡SÍ!

Encontrado en 4 pasos.

1. Empiezas en el primer elemento (índice 0).
2. Comparas el elemento actual con el valor que buscas.
3. Si son iguales, devuelves el índice o `true`.
4. Si no son iguales, pasas al siguiente elemento.
5. Repites hasta encontrarlo o llegar al final del array.
6. Si llegas al final y no lo encontraste, devuelves -1 o `false`.

PSEUDOCÓDIGO

```
función busquedaLineal(array, objetivo):  
    PARA cada i desde 0 hasta tamaño(array) - 1:
```

```

13     SI array[i] == objetivo:
14         return i
15     return -1
16
17 CÓDIGO C++
18
19
20 #include <iostream>
21 #include <vector>
22 using namespace std;
23
24 int linearSearch(const vector<int>& arr, int target) {
25     for (int i = 0; i < arr.size(); i++) {
26         if (arr[i] == target) {
27             return i; // Encontrado, devolvemos el índice
28         }
29     }
30     return -1; // No encontrado
31 }
32
33 int main() {
34     vector<int> data = {10, 50, 30, 70, 80, 60, 20, 90, 40};
35     int target = 20;
36
37     int result = linearSearch(data, target);
38
39     if (result != -1)
40         cout << "Elemento encontrado en el índice: " << result <<
41             endl;
42     else
43         cout << "Elemento no encontrado." << endl;
44
45     return 0;
46 }
47
48 COMPLEJIDAD (Big-O)
49
50     Caso            Tiempo            Razón
51
52     Best            O(1)                El elemento es el primero
53     Average         O(n)                Tienes que revisar la mitad (n/2)
54     Worst           O(n)                Está al final o no existe
55     Espacio         O(1)                No usa memoria extra

```

En entrevistas

Cuándo usar: - Datasets pequeños ($n < 50$). - Datos desordenados. - Búsquedas únicas (one-off). - Estructuras sin acceso aleatorio (como Linked Lists).

Strings: Linear Search es la base para encontrar un caracter en un string o una subcadena (algoritmo naive).

Eficiencia: Si vas a hacer MUCHAS búsquedas sobre el mismo conjunto de datos, ¡no uses Linear Search! Mejor ordena los datos y usa Binary Search, o guárdalos en un Hash Map.

Sentinel Search: Una pequeña optimización que elimina una de las dos comparaciones del loop (la que revisa si llegaste al final).



Resumen rápido

- Busca uno por uno.
- Muy simple de implementar.
- No requiere orden previo.
- Tiempo lineal $O(n)$.
- Memoria constante $O(1)$.
- Ineficiente para datasets grandes.

Capítulo 33

Binary Search (Búsqueda Binaria)

¿Qué es?

El juego de adivinar el número entre 1 y 100.

Dices "50" y te dicen: "¡Más alto!". Ahora sabes que el número NO está entre 1 y 50. ¡Acabas de descartar la MITAD de las posibilidades en un solo paso!

Luego dices "75", te dicen: "¡Más bajo!". Ahora sabes que está entre 51 y 74. Vuelves a descartar la mitad.

En solo 7 intentos puedes adivinar cualquier número entre 128 opciones. Es increíblemente rápido porque el problema se encoge exponencialmente en cada paso.

Dicho de otra forma

Binary Search es un algoritmo de búsqueda eficiente que funciona sobre un conjunto de datos ORDENADO.

En lugar de revisar uno por uno, siempre revisa el elemento del medio. Si el objetivo es menor al medio, busca en la mitad izquierda. Si es mayor, busca en la derecha.

Este enfoque de "Divide y Vencerás" reduce la complejidad de tiempo a $O(\log n)$, lo cual es ideal para datasets masivos.

Diagrama

Buscar 23 en array ordenado:

[1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25]

idx: 0 1 2 3 4 5 6 7 8 9 10 11 12

Paso 1: izq=0, der=12, mid=6.

arr[6] es 13. ¿13 < 23? SÍ.

Nuevo rango: [7...12] (la derecha)

Paso 2: izq=7, der=12, mid=9.

arr[9] es 19. ¿19 < 23? SÍ.

Nuevo rango: [10...12]

Paso 3: izq=10, der=12, mid=11.
arr[11] es 23. ¿23 == 23? ¡SÍ!

Solo 3 pasos para 13 elementos.

1 Requisito: El array DEBE estar ordenado de menor a mayor.

2 Pasos:

3 1. Define dos punteros: `izq` al inicio y `der` al final.

4 2. Mientras `izq <= der`:

5 - Calcula el punto medio: `mid = izq + (der - izq) / 2`.

6 - Si `arr[mid] == objetivo`, ¡terminaste! Devuelve el índice.

7 - Si `arr[mid] < objetivo`, mueve `izq = mid + 1`.

8 - Si `arr[mid] > objetivo`, mueve `der = mid - 1`.

9 3. Si sales del loop sin encontrarlo, devuelve -1.

10 PSEUDOCÓDIGO

11 función busquedaBinaria(arr, x):

12 izq = 0

13 der = arr.tamaño - 1

14 MIENTRAS izq <= der:

15 mid = izq + (der - izq) / 2

16 SI arr[mid] == x: **return** mid

17 SI arr[mid] < x: izq = mid + 1

18 SINO: der = mid - 1

19 **return** -1

20 CÓDIGO C++

21 **#include** <iostream>

22 **#include** <vector>

23 **using namespace** std;

24 **int** binarySearch(**const** vector<**int**>& arr, **int** target) {

25 **int** left = 0;

26 **int** right = arr.size() - 1;

27 **while** (left <= right) {

28 // Usamos esta fórmula para evitar overflow de enteros

29 **int** mid = left + (right - left) / 2;

30 **if** (arr[mid] == target) **return** mid;

31 **if** (arr[mid] < target)

32 left = mid + 1; // Buscar en la derecha

33 **else**

34 right = mid - 1; // Buscar en la izquierda

```

46     }
47     return -1;
48 }
49
50 int main() {
51     vector<int> sorted_data = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
52     int target = 15;
53
54     int res = binarySearch(sorted_data, target);
55     if(res != -1) cout << "Encontrado en idx: " << res << endl;
56
57     return 0;
58 }
59
60 COMPLEJIDAD (Big-O)
61
62
63     Caso            Tiempo            Razón
64
65     Best            O(1)              El elemento está justo al medio
66     Average         O(log n)          Se divide el problema a la mitad
67     Worst           O(log n)          No existe o queda 1 solo elemento
68     Espacio         O(1)              Versión iterativa

```

En entrevistas

Off-by-one errors: El bug más común es usar 'left < right' en lugar de 'left <= right'. ¡Ten cuidado!

Overflow Bug: NUNCA uses 'mid = (left + right) / 2'. Si left y right son muy grandes, su suma puede superar el límite de un 'int'. Usa siempre: 'left + (right - left) / 2'.

Rotated Sorted Array: Un problema clásico de Google es buscar en un array ordenado que fue "rotado" (ej: [4,5,6,1,2,3]). Sigue siendo Binary Search pero con lógica extra.

Binary Search on Answer: No solo sirve para buscar en arrays. Si te preguntan "¿cuál es el mínimo valor de X que cumple esta condición?", puedes hacer Binary Search sobre el RANGO de posibles valores de X.

Resumen rápido

- Solo funciona con datos ordenados.
- Divide el espacio de búsqueda a la mitad en cada paso.
- Tiempo logarítmico $O(\log n)$: Súper veloz.
- Versión iterativa usa $O(1)$ memoria.
- Previene overflow usando la fórmula correcta de 'mid'.
- Es una de las herramientas más poderosas para optimizar búsquedas.

Patrones de Entrevista

Capítulo 34

👉 Two Pointers (Dos Punteros)

🤖 ¿Qué es?

Dos personas en extremos opuestos de un pasillo ... caminando hacia el centro para encontrarse. O un corredor lento y uno rápido en la misma pista

En lugar de usar un solo índice para recorrer un array (un loop simple), usas DOS. Estos punteros se mueven de forma coordinada para resolver problemas de búsqueda o comparación mucho más rápido que la fuerza bruta.

Es uno de los patrones más útiles para entrevistas técnicas porque suele reducir soluciones de $O(n^2)$ a $O(n)$.

💬 Dicho de otra forma

El patrón Two Pointers utiliza dos variables de índice para recorrer una estructura de datos lineal (array, string, linked list).

Existen dos variantes principales: 1. Opposite Ends: Los punteros empiezan en los extremos y se mueven hacia el centro. (Ej: Two Sum en array ordenado, Palíndromos). 2. Fast & Slow: Un puntero avanza más rápido que el otro. (Ej: Detectar ciclos en Linked List, encontrar el medio).

🧠 Diagrama

Problema: Encontrar par que sume 10 en array ordenado.

[1, 3, 4, 6, 8, 9]

L R -> 1 + 9 = 10. ¡ENCONTRADO!

Si la suma fuera menor a 10, moveríamos L a la derecha.

Si la suma fuera mayor a 10, moveríamos R a la izquierda.

⚙️ ¿Cómo funciona?

Opposite Ends: 1. Inicializas 'left = 0' y 'right = n - 1'. 2. Mientras 'left < right': - Evalúas la condición con 'arr[left]' y 'arr[right]'. - Si se cumple la meta, terminas. - Si no, decides qué puntero mover según la lógica del problema.

Fast & Slow (Floyd's Algorithm): 1. Inicializas 'slow = start' y 'fast = start'. 2. En cada paso: 'slow' avanza 1, 'fast' avanza 2. 3. Si 'fast' alcanza el final, no hay ciclo. 4. Si 'slow == fast', ¡hay un ciclo!

PSEUDOCÓDIGO

```
// Two Sum en Array Ordenado
función tieneSuma(arr, meta):
    izq = 0
    der = arr.tamaño - 1
    MIENTRAS izq < der:
        sumaActual = arr[izq] + arr[der]
        SI sumaActual == meta: return VERDADERO
        SI sumaActual < meta: izq++
        SINO: der--
    return FALSO
```

CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5 using namespace std;
6
7 // Verificar si un string es palíndromo -> O(n) tiempo, O(1)
   espacio
8 bool isPalindrome(string s) {
9     int left = 0;
10    int right = s.length() - 1;
11
12    while (left < right) {
13        if (s[left] != s[right]) return false;
14        left++;
15        right--;
16    }
17    return true;
18 }
19
20 int main() {
21     string word = "reconocer";
22     cout << "¿Es palíndromo? " << (isPalindrome(word) ? "SÍ" : "NO"
23         ) << endl;
24
25     return 0;
26 }
```

Complejidad Big-O

Caso	Tiempo	Espacio	
O(n)	O(1)	Fuerza Bruta	$O(n^2)$ O(1)

Two Pointers

* La magia del patrón es que evitas el segundo loop anidado.

En entrevistas

Resumen rápido

- Dos índices moviéndose coordinados.
- Reduce $O(n^2)$ a $O(n)$.
- Memoria constante $O(1)$ casi siempre.
- Variantes: Opuestos (extremos $\rightarrow$ centro) y Fast/Slow.
- Requisito común: Array ordenado.
- Indispensable para optimizar búsquedas de pares o ciclos.

Capítulo 35

□ Sliding Window (Ventana Deslizante)

🤖 ¿Qué es?

Una ventana de tren donde ves paisajes mientras el tren avanza. La ventana tiene un tamaño fijo (por ejemplo, 3 metros de ancho).

Al avanzar, entra nuevo paisaje por un lado y sale el paisaje viejo por el otro. No tienes que redibujar todo el paisaje cada vez, ¡solo actualizas los bordes!

En programación, este patrón sirve para procesar sub-arreglos o sub-cadenas sin tener que recalculas todo desde cero en cada paso, lo cual ahorra muchísimo tiempo.

💬 Dicho de otra forma

El patrón Sliding Window convierte un problema de loops anidados $O(n*k)$ en un problema de un solo loop $O(n)$.

Se usa cuando te piden algo sobre elementos CONTIGUOS de un array o string. Existen dos tipos: 1. Fixed Size: La ventana siempre mide K elementos. 2. Variable Size: La ventana crece o se encoge según una condición (usualmente usando dos punteros).

🤖 Diagrama

Array: [1, 3, 2, 6, -1, 4]

Meta: Suma de cada ventana de tamaño 3.

Paso 1: [1, 3, 2] , 6, -1, 4 -> Suma = 6

Paso 2: 1, [3, 2, 6] , -1, 4 -> Suma = 11 (6 - 1 + 6)

Paso 3: 1, 3, [2, 6, -1] , 4 -> Suma = 7 (11 - 3 + (-1))

¡Solo restas el que sale y sumas el que entra!

⚙️ ¿Cómo funciona?

Fixed Size Window: 1. Calculas el resultado de la primera ventana (índices 0 a K-1). 2. Empiezas un loop desde el índice K hasta el final. 3. En cada paso: -

Sumas el nuevo elemento ('arr[i]'). - Restas el elemento que sale ('arr[i - K]'). - Actualizas el resultado máximo/mínimo.

Variable Size Window: 1. Usas dos punteros: 'start' y 'end' (ambos en 0). 2. Mueves 'end' para expandir la ventana y cumplir una condición. 3. Si la condición se rompe, mueves 'start' para encoger la ventana.

PSEUDOCÓDIGO

```
función sumaMaxima(arr, k):
    sumaActual = suma de los primeros k elementos
    maximaSuma = sumaActual
    PARA i desde k hasta arr.tamaño - 1:
        sumaActual = sumaActual + arr[i] - arr[i - k]
        maximaSuma = max(maximaSuma, sumaActual)
    return maximaSuma
```

CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 // Máxima suma de un subarray de tamaño K -> O(n)
7 int maxSubarraySum(vector<int>& arr, int k) {
8     if (arr.size() < k) return -1;
9
10    int window_sum = 0;
11    for (int i = 0; i < k; i++) window_sum += arr[i];
12
13    int max_sum = window_sum;
14
15    for (int i = k; i < arr.size(); i++) {
16        // Deslizar la ventana: sumar el nuevo, restar el viejo
17        window_sum += arr[i] - arr[i - k];
18        max_sum = max(max_sum, window_sum);
19    }
20
21    return max_sum;
22 }
23
24 int main() {
25     vector<int> data = {2, 1, 5, 1, 3, 2};
26     int k = 3;
```

```
27     cout << "Máxima suma (k=3): " << maxSubarraySum(data, k) <<
        endl; // 9 (5+1+3)
28     return 0;
29 }
```

Capítulo 36

🌸 Introducción a Dynamic Programming (DP)

🧠 ¿Qué es?

Un estudiante inteligente que guarda sus apuntes .

Imagina que en un examen te preguntan 50 veces cuánto es 123×456 . En lugar de hacer la multiplicación cada vez (y perder tiempo), la haces la primera vez, anotas el resultado en tu cuaderno, y las siguientes 49 veces ¡solo lo consultas!

Dynamic Programming es simplemente eso: guardar resultados de sub-problemas ya resueltos para no repetir trabajo innecesariamente. Es el arte de intercambiar un poco de memoria por mucho tiempo.

💬 Dicho de otra forma

Dynamic Programming (DP) es una técnica de optimización para resolver problemas complejos dividiéndolos en sub-problemas más pequeños.

Para usar DP, el problema debe cumplir dos condiciones: 1. Overlapping Sub-problems: El mismo sub-problema se calcula varias veces (como en Fibonacci). 2. Optimal Substructure: La solución óptima del problema global se puede construir a partir de las soluciones óptimas de sus partes.

🕸 Diagrama

Sin DP (Calculas lo mismo muchas veces):

```
          fib(5)
         /    \
      fib(4)   fib(3)
     /  \   /  \
  fib(3) fib(2) fib(2) fib(1)
   /  \
 fib(2) fib(1)
```

¡Nota cómo fib(3) y fib(2) se repiten!

Con DP (Memoization):

Calculas fib(3) una vez, lo guardas, y cuando fib(4) lo necesite, simplemente lo lee de la memoria.

 ¿Cómo funciona?

Existen dos formas de implementar DP:

1. Top-Down (Memoization): - Empiezas con el problema grande. - Lo divides recursivamente. - Guardas el resultado en un Hash Map o Array antes de regresarlo.
2. Bottom-Up (Tabulation): - Empiezas con los casos más pequeños (base cases). - Llenas una tabla (array) paso a paso hasta llegar al resultado. - Generalmente se hace con un loop iterativo.

PSEUDOCÓDIGO

```
// Fibonacci Sin DP:  $O(2^n)$  - Pésimo
función fib(n):
    SI n <= 1: return n
    return fib(n-1) + fib(n-2)

// Fibonacci Con DP (Memo):  $O(n)$  - Excelente
función fib(n, memo):
    SI n EN memo: return memo[n]
    SI n <= 1: return n
    memo[n] = fib(n-1, memo) + fib(n-2, memo)
    return memo[n]
```

CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Fibonacci sin DP (muy lento para n > 40)
6 long long fibNaive(int n) {
7     if (n <= 1) return n;
8     return fibNaive(n - 1) + fibNaive(n - 2);
9 }
10
11 // Fibonacci con DP (Tabulation - Bottom Up)
12 long long fibDP(int n) {
13     if (n <= 1) return n;
14     vector<long long> dp(n + 1);
15     dp[0] = 0;
16     dp[1] = 1;
17     for (int i = 2; i <= n; i++) {
18         dp[i] = dp[i - 1] + dp[i - 2];
19     }
```

```

20     return dp[n];
21 }
22
23 int main() {
24     int n = 50;
25     cout << "Fibonacci de " << n << " es: " << fibDP(n) << endl;
26     return 0;
27 }

```

Complejidad Big-O

Enfoque	Tiempo	Espacio	Fuerza Bru-
ta	$O(2^n)$	$O(n)$ (stack)	ta
		DP (Memo/Tab)	$O(n)$ $O(n)$ (tabla)

* La DP reduce la complejidad de exponencial a lineal en muchos casos.

En entrevistas

Resumen rápido

- DP = Recursión + Memoria.
- No calcules lo mismo dos veces.
- Overlapping Subproblems: La clave para usar DP.
- Top-Down: Fácil de pensar (recursivo).
- Bottom-Up: Más eficiente (iterativo).
- Esencial para problemas de optimización.

Capítulo 37

💡 DP - Memoization (Top-Down)

🤖 ¿Qué es?

Un chef que anota recetas nuevas en un cuaderno .

Imagina que estás cocinando platillos complejos. La primera vez que haces una salsa secreta, tardas mucho tiempo y esfuerzo. En lugar de olvidar cómo la hiciste, anotas los pasos y el resultado en tu cuaderno.

La siguiente vez que alguien pide ese platillo, no vuelves a experimentar; simplemente abres tu cuaderno, lees la receta y ¡listo! Tienes el resultado al instante.

En programación, Memoization es el cuaderno donde guardas el resultado de una función para ciertos argumentos, para no tener que volver a ejecutar toda la lógica si te vuelven a pedir lo mismo.

💬 Dicho de otra forma

Memoization es la técnica de DP basada en Recursión (Top-Down).

Empiezas con el problema original (el más grande) y lo vas rompiendo en partes pequeñas. Antes de regresar el valor de una función, lo guardas en una estructura de datos (generalmente un Array o un Hash Map) donde la Key son los argumentos de la función.

Es muy natural de implementar si ya tienes una solución recursiva, ya que solo requiere agregar el paso de "Revisar cuaderno" y "Anotar en cuaderno".

🧠 Diagrama

Llamada: $f(5)$

1. ¿ $f(5)$ está en el cuaderno?
 - SÍ: Regresa el valor guardado. $O(1)$
 - NO:
 - a. Calcula $f(5)$ haciendo $f(4) + f(3)$.
 - b. Guarda el resultado: `cuaderno[5] = resultado`.
 - c. Regresa el resultado.

¡Esto convierte un árbol de recursión gigante en un camino lineal!

⚙️ ¿Cómo funciona?

Para cualquier función recursiva: 1. Pasa una estructura 'memo' como parámetro (o úsala global). 2. CASO BASE: Define cuándo detener la recursión. 3. REVISAR MEMO: Si el resultado ya está calculado, regresa-lo. 4. CALCULAR Y GUARDAR: Calcula el valor, guárdalo en 'memo' y regresa-lo.

PSEUDOCÓDIGO

```
función minMonedas(monto, monedas, memo):
    SI monto == 0: return 0
    SI monto < 0: return INFINITO
    SI monto EN memo: return memo[monto]

    minimo = INFINITO
    PARA cada moneda EN monedas:
        resultado = minMonedas(monto - moneda, monedas, memo)
        SI resultado != INFINITO:
            minimo = min(minimo, resultado + 1)

    memo[monto] = minimo
    return minimo
```

CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <unordered_map>
4 #include <algorithm>
5 using namespace std;
6
7 // Problema: Coin Change (Mínimo de monedas para una cantidad)
8 // Tiempo: O(monto * n_monedas)
9 // Espacio: O(monto) para el memo
10 int solve(int amount, vector<int>& coins, unordered_map<int, int>& memo) {
11     if (amount == 0) return 0;
12     if (amount < 0) return -1;
13     if (memo.count(amount)) return memo[amount];
14
15     int min_coins = 1e9;
16     bool possible = false;
17
18     for (int coin : coins) {
19         int res = solve(amount - coin, coins, memo);
```

```

20     if (res >= 0) {
21         min_coins = min(min_coins, res + 1);
22         possible = true;
23     }
24 }
25
26 return memo[amount] = (possible ? min_coins : -1);
27 }
28
29 int main() {
30     vector<int> coins = {1, 2, 5};
31     int amount = 11;
32     unordered_map<int, int> memo;
33
34     cout << "Mínimo de monedas para " << amount << ": "
35           << solve(amount, coins, memo) << endl; // 3 (5+5+1)
36
37     return 0;
38 }

```

Complejidad Big-O

Aspecto	Con Memoization	Tiempo
O(Estados Únicos * Trabajo por Estado)	O(Estados Únicos) + Stack de recursión	
* Para Coin Change: $O(\text{Amount} * N_{\text{coins}})$ tiempo. Mucho mejor que $O(N_{\text{coins}}^{\text{Amount}})$ de la fuerza bruta.		



En entrevistas



Resumen rápido

- Top-Down: Del problema grande a los pequeños.
- Recursión + Diccionario/Array de resultados.
- Anota resultados para no repetir trabajo.
- Transforma complejidad exponencial en lineal/polinomial.
- Ideal para cuando la estructura del problema es un árbol.

Capítulo 38

DP - Tabulation (Bottom-Up)

¿Qué es?

Construir una pared de ladrillos desde el suelo hacia arriba.

En lugar de empezar por el techo (el problema grande) y tratar de adivinar qué ladrillos necesitas abajo (recursión), empiezas poniendo los primeros ladrillos en la base (los casos base que ya conoces).

Una vez que tienes la primera fila, puedes poner la segunda encima usando la base como apoyo. Vas llenando tu tabla de resultados paso a paso hasta que llegas a la altura que querías (el resultado final).

Dicho de otra forma

Tabulation es la técnica de DP basada en Iteración (Bottom-Up).

Se llama así porque literalmente llenas una tabla (un array de 1D o 2D) con los resultados de los sub-problemas, empezando por los más pequeños. Cada nueva entrada de la tabla se calcula usando los valores que ya están guardados en las posiciones anteriores.

Es generalmente más eficiente que Memoization porque evita el "overhead" (sobrecosto) de las llamadas recursivas y no corre el riesgo de causar un Stack Overflow.

Diagrama

Queremos `fib(5)`.

Tabla DP de tamaño 6: [`\_ , \_ , \_ , \_ , \_ , \_`]

1. Llenamos bases: `dp[0]=0, dp[1]=1`
[`0, 1, \_ , \_ , \_ , \_`]

2. Llenamos el resto usando `dp[i] = dp[i-1] + dp[i-2]`:
`i=2`: [`0, 1, 1, \_ , \_ , \_`]
`i=3`: [`0, 1, 1, 2, \_ , \_`]
`i=4`: [`0, 1, 1, 2, 3, \_`]
`i=5`: [`0, 1, 1, 2, 3, 5`] -> Resultado final

¿Cómo funciona?

1. Inicializa una tabla (array o matriz) con valores default. 2. Define los casos base en las primeras posiciones de la tabla. 3. Escribe un loop (o varios) que recorra la tabla. 4. En cada paso, aplica la lógica para llenar la celda actual usando celdas anteriores. 5. Regresa el valor de la última celda relevante.

PSEUDOCÓDIGO

```
función minMonedasTab(monto, monedas):
    tabla = array de tamaño (monto + 1) con INFINITO
    tabla[0] = 0

    PARA cada moneda EN monedas:
        PARA i desde moneda hasta monto:
            SI tabla[i - moneda] != INFINITO:
                tabla[i] = min(tabla[i], tabla[i - moneda] + 1)

    return tabla[monto]
```

CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 // Problema: Climbing Stairs (¿De cuántas formas puedes subir N
7 // si puedes dar pasos de 1 o 2?) -> O(n) tiempo, O(n) espacio
8 int climbStairs(int n) {
9     if (n <= 2) return n;
10
11     vector<int> dp(n + 1);
12     dp[1] = 1;
13     dp[2] = 2;
14
15     for (int i = 3; i <= n; i++) {
16         dp[i] = dp[i - 1] + dp[i - 2];
17     }
18
19     return dp[n];
20 }
21
22 // OPTIMIZACIÓN DE ESPACIO: O(n) -> O(1)
```

```

23 // Como solo necesitamos los últimos 2 valores, no hace falta el
    array.
24 int climbStairsOptimized(int n) {
25     if (n <= 2) return n;
26     int prev2 = 1, prev1 = 2;
27     for (int i = 3; i <= n; i++) {
28         int current = prev1 + prev2;
29         prev2 = prev1;
30         prev1 = current;
31     }
32     return prev1;
33 }
34
35 int main() {
36     int stairs = 5;
37     cout << "Formas de subir " << stairs << " escalones: "
38         << climbStairsOptimized(stairs) << endl; // 8
39     return 0;
40 }

```

Complejidad Big-O

Aspecto	Con Tabulation	Tiempo
O(Tamaño de la Tabla * Trabajo por Celda)	Espacio	O(Tamaño de la Tabla)
-> A veces reducible a O(1)		
* Al ser iterativo, el tiempo es muy predecible y constante.		

En entrevistas

Resumen rápido

- Bottom-Up: De lo simple a lo complejo.
- Iterativo: Sin recursión, sin stack overflow.
- Llenas una tabla paso a paso.
- Más eficiente en tiempo y memoria.
- Permite optimizar espacio fácilmente (usando variables en vez de array).
- Es la forma "profesional" de entregar una solución DP.

Capítulo 39

Backtracking (Vuelta Atrás)

¿Qué es?

Resolver un laberinto con un lápiz.

Sigues un camino con la punta del lápiz hasta que llegas a un callejón sin salida. En lugar de rendirte, borras lo que hiciste (vuelta atrás) hasta la última bifurcación y pruebas el otro camino que dejaste pendiente.

Backtracking es probar TODAS las posibilidades una por una, pero con la inteligencia de "deshacer" tus pasos cuando te das cuenta de que una ruta no lleva a la solución. Es una búsqueda exhaustiva pero controlada.

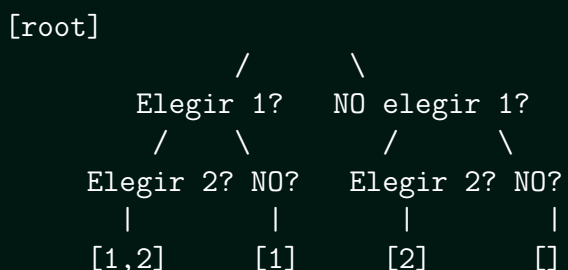
Dicho de otra forma

Backtracking es una técnica algorítmica para resolver problemas de forma recursiva, intentando construir una solución incrementalmente.

Se basa en la exploración de un "Árbol de Decisiones". Si en algún punto la solución parcial no cumple con las restricciones del problema (se vuelve inválida), el algoritmo descarta esa rama ("pruning") y regresa al estado anterior para intentar otra opción.

Es la base para resolver problemas de combinatoria, permutaciones y juegos como el Sudoku o el Ajedrez.

Diagrama



¡Cada nodo es una decisión! Backtracking recorre este árbol usando DFS.

⚙️ ¿Cómo funciona?

Casi todos los problemas de Backtracking siguen este patrón:

1. Caso Base: ¿Ya terminamos de construir la solución? - SÍ: Guárdala o imprímela. Regresa.
2. Opciones: Para cada opción disponible en este nivel: - ¿Es válida la opción? (Restricciones). - SÍ:
 - a. Hacer Elección: Modifica el estado actual.
 - b. Recursión: Llama a la función para el siguiente nivel.
 - c. Deshacer Elección (BACKTRACK): Regresa el estado como estaba.

PSEUDOCÓDIGO

```
función backtrack(estado, opciones):
    SI estado es meta:
        imprimir(estado)
        return

    PARA cada o EN opciones:
        SI esValida(o, estado):
            agregar o a estado // "Hacer"
            backtrack(estado, opciones\_restantes)
            quitar o de estado // "Deshacer" (La clave )
```

CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 using namespace std;
5
6 // Problema: Generar todas las permutaciones de un string -> O(n!)
7 void permute(string& s, int l, int r) {
8     // Caso Base
9     if (l == r) {
10         cout << s << endl;
11         return;
12     }
13
14     for (int i = l; i <= r; i++) {
15         // 1. Hacer Elección (Swap)
16         swap(s[l], s[i]);
17
18         // 2. Recursión (Ir al siguiente nivel)
19         permute(s, l + 1, r);
20
21         // 3. Deshacer Elección (Backtrack)
```

```

22         // Regresamos el string a su estado original para la sig.
           iteración
23         swap(s[l], s[i]);
24     }
25 }
26
27 int main() {
28     string str = "ABC";
29     cout << "Permutaciones de ABC:" << endl;
30     permute(str, 0, str.length() - 1);
31     return 0;
32 }

```

Complejidad Big-O

Problema	Tiempo	Típico	Razón
Permutaciones	$O(n!)$	$n * (n-1) * (n-2) \dots$	Subconjuntos $O(2^n)$ Cada elemento se elige o no
N-Queens	$O(n!)$	Menos gracias al pruning	Sudoku $O(9^{(n*n)})$
Fuerza bruta pura			(peor caso)

* El espacio suele ser $O(n)$ por la profundidad del stack de recursión.



En entrevistas



Resumen rápido

- DFS + Deshacer cambios.
- Explora todas las soluciones posibles.
- Úsalo para permutaciones, combinaciones y laberintos.
- Complejidad alta (Exponencial/Factorial).
- Pruning: Cortar ramas que no sirven para ganar velocidad.
- Esencial dominar el template: Hacer -> Recursión -> Deshacer.

Capítulo 40

12 34 Bit Manipulation (Manipulación de Bits)

¿Qué es?

Los interruptores de luz de tu casa.

Cada bit es como un interruptor: 1 significa "encendido" y 0 significa "apagado". Manipular bits es jugar con esos interruptores usando reglas lógicas: - AND: Solo enciende si AMBOS están encendidos. - OR: Enciende si AL MENOS UNO está encendido. - XOR: Solo enciende si EXACTAMENTE UNO está encendido.

Es la forma más profunda y rápida de hablarle a la computadora, directamente en su lenguaje natural de ceros y unos.

Dicho de otra forma

Bit Manipulation consiste en aplicar operaciones lógicas (AND, OR, XOR, NOT) y de desplazamiento (Shifts) sobre los bits individuales de un número entero.

A nivel de hardware, estas operaciones son las más rápidas que un procesador puede realizar. Usar bits permite ahorrar muchísima memoria y acelerar algoritmos que de otra forma usarían estructuras más pesadas.

Diagrama

Sean $A = 5$ (101) y $B = 3$ (011)

AND ($\&$): $101 \& 011 = 001$ (1)

OR ($|$): $101 | 011 = 111$ (7)

XOR ($\wedge$): $101 \wedge 011 = 110$ (6) <-- ¡Nota: $1 \wedge 1 = 0$!

Left Shift ($\ll$): $5 \ll 1 = 1010$ (10) (Multiplicar por 2)

Right Shift ($\gg$): $5 \gg 1 = 10$ (2) (Dividir entre 2)

¿Cómo funciona?

1. $x \wedge x = 0$: Cualquier número XOR consigo mismo da cero. (Base del problema "Single Number"). 2. $x \wedge 0 = x$: Cualquier número XOR cero se queda igual. 3. $n \& (n - 1)$: ¡Truco de Google! Elimina el bit encendido más a la derecha. Si el

resultado es 0, entonces 'n' era una potencia de 2. 4. $i \& 1$: Si da 0 es par, si da 1 es impar. (Más rápido que $i \% 2$). 5. $n \gg 1$: Divide entre 2 (truncando). 6. $1 \ll n$: Calcula 2 elevado a la potencia 'n'.

PSEUDOCÓDIGO

```
función contarBits(n):
    contador = 0
    MIENTRAS n > 0:
        n = n & (n - 1) // Eliminar el bit más a la derecha
        contador++
    return contador
```

CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Problema: Encontrar el número que no se repite
6 // en un array donde todos los demás aparecen 2 veces.
7 int findSingleNumber(vector<int>& nums) {
8     int res = 0;
9     for (int x : nums) {
10         res ^= x; // Los duplicados se cancelan:  $x \oplus x = 0$ 
11     }
12     return res;
13 }
14
15 // Verificar si es potencia de 2
16 bool isPowerOfTwo(int n) {
17     return n > 0 && (n & (n - 1)) == 0;
18 }
19
20 int main() {
21     vector<int> data = {4, 1, 2, 1, 2};
22     cout << "Número único: " << findSingleNumber(data) << endl; //
23         4
24
25     int val = 16;
26     cout << "¿16 es potencia de 2? " << isPowerOfTwo(val) << endl;
27     // 1
28
29     return 0;
30 }
```

Complejidad Big-O

Operación	Tiempo	Espacio	Bitwise (&, ,^,~)
O(1)	O(1)	O(1)	O(1)

* Son las operaciones más eficientes posibles en computación.

En entrevistas

Resumen rápido

- Operaciones a nivel de ceros y unos.
- Súper rápidas O(1).
- AND (&), OR (|), XOR (^), NOT (~).
- Shifts: « (x2), » (/2).
- $n \& (n-1)$ = Truco para potencias de 2.
- XOR cancela duplicados ($x \hat{ } x = 0$).
- Ideal para problemas de eficiencia extrema.

Capítulo 41

Recursión y Memoization

¿Qué es?

Las muñecas rusas Matryoshka .

Abres una muñeca grande y adentro hay otra igual pero más pequeña. Sigues abriendo muñecas hasta que llegas a la más pequeñita de todas, la cual ya no tiene nada adentro. Ese es tu "punto de parada".

Recursión es una función que se llama a sí misma para resolver una versión más pequeña del mismo problema. Sigues bajando niveles hasta llegar al "Caso Base" (la muñeca más chica), y luego vas regresando hacia arriba con las respuestas de cada nivel.

Dicho de otra forma

La Recursión es un paradigma de programación donde una función se define en términos de sí misma.

Para que una función recursiva sea correcta, DEBE tener: 1. Caso Base: La condición de salida que detiene las llamadas. 2. Caso Recursivo: La lógica que rompe el problema en una parte más pequeña y hace la auto-llamada.

Si olvidas el Caso Base, la función se llamará infinitamente hasta agotar la memoria del sistema (Stack Overflow).

Diagrama

Llamada inicial: `fact(3)`

```
[ fact(1) ] -> retorna 1 (¡Caso Base!)
```

```
[ fact(2) ] -> espera a fact(1)... recibe 1, retorna 2 * 1 = 2
```

```
[ fact(3) ] -> espera a fact(2)... recibe 2, retorna 3 * 2 = 6
```

Resultado Final: 6

¿Cómo funciona?

- Recursión: Más elegante, fácil de leer para problemas de árboles o gráficos. Usa memoria en el "Call Stack" para cada llamada. - Iteración (Loops): Más eficiente en memoria ($O(1)$ espacio extra). A veces es más difícil de pensar para problemas complejos.

Memoization (La mejora): Como vimos en DP, si una función recursiva se llama con los mismos argumentos muchas veces, guardamos el resultado en un caché (Memoization) para que la segunda llamada sea $O(1)$.

PSEUDOCÓDIGO

```
función factorial(n):
    SI n <= 1: return 1 // Caso Base
    return n * factorial(n - 1) // Caso Recursivo
```

CÓDIGO C++

```
1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // Recursión Simple
6 long long factorial(int n) {
7     if (n <= 1) return 1;
8     return n * factorial(n - 1);
9 }
10
11 // Recursión con Memoization (Fibonacci)
12 long long fibMemo(int n, vector<long long>& memo) {
13     if (n <= 1) return n;
14     if (memo[n] != -1) return memo[n]; // Revisar caché
15
16     return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
17 }
18
19 int main() {
20     cout << "Factorial de 5: " << factorial(5) << endl; // 120
21
22     int n = 40;
23     vector<long long> memo(n + 1, -1);
24     cout << "Fibonacci de " << n << ": " << fibMemo(n, memo) <<
25         endl;
26
27     return 0;
28 }
```

 Complejidad Big-O

Algoritmo	Tiempo	(Normal)	Con	Memoization
		Factorial	$O(n)$	$O(n)$ Fibonacci
$O(2^n)$	$O(n)$	Espacio	$O(n)$ (Stack)	$O(n)$ (Stack + Cache)

 En entrevistas Resumen rápido

- Función que se llama a sí misma.
- Requiere Caso Base y Caso Recursivo.
- Úsala para árboles, grafos y problemas de "Divide y Vencerás".
- Memoization: El truco para que la recursión sea ultra rápida.
- Cuidado con el Stack Overflow en inputs grandes.
- Cada nivel de recursión = $O(1)$ de espacio extra en el stack.